



Laboratorio di Ingegneria dell'Informazione – Modulo 3

Linguaggio C e Applicazioni

Prof. Enrico Paolini
A.A. 2025/2026

Outline

- 1 Introduzione
- 2 Variabili e costanti
- 3 Input/output di base
- 4 Rappresentazione binaria dei dati
- 5 Operatori
- 6 Conversioni di tipo
- 7 Array e puntatori
- 8 Funzioni
- 9 Controllo del flusso
- 10 Input/output da file
- 11 Variabili aleatorie in C
- 12 Numeri complessi in C
- 13 Elaborazione in virgola fissa

Outline

- 1 Introduzione
- 2 Variabili e costanti
- 3 Input/output di base
- 4 Rappresentazione binaria dei dati
- 5 Operatori
- 6 Conversioni di tipo
- 7 Array e puntatori
- 8 Funzioni
- 9 Controllo del flusso
- 10 Input/output da file
- 11 Variabili aleatorie in C
- 12 Numeri complessi in C
- 13 Elaborazione in virgola fissa

Un primo esempio di programma C

"Hello World!"

```
/* Program to print ``Hello World!`` on the screen */  
  
#include <stdio.h>  
int main(void) {  
    printf("Hello World!\n");  
    return(0);  
}
```

- Da notare:
 - ▶ Uso di funzioni (una sola in questo caso, `main`)
 - ▶ Inclusione di funzioni della libreria standard I/O
 - ▶ Linguaggio case-sensitive
 - ▶ Uso di commenti
 - ▶ Corretta indentazione del codice

Outline

- 1 Introduzione
- 2 Variabili e costanti**
- 3 Input/output di base
- 4 Rappresentazione binaria dei dati
- 5 Operatori
- 6 Conversioni di tipo
- 7 Array e puntatori
- 8 Funzioni
- 9 Controllo del flusso
- 10 Input/output da file
- 11 Variabili aleatorie in C
- 12 Numeri complessi in C
- 13 Elaborazione in virgola fissa



Tipi di dato di base

- I dati manipolati in un programma C possono essere classificati in **variabili** e **costanti**
- **Operatori**: specificano le operazioni eseguite sui dati
- **Espressioni**: combinano dati e operatori, per generare nuovi valori
- In C esistono quattro tipi di dato di base: `int`, `char`, `float`, `double`
 - ▶ `int`: numero intero con segno
 - ▶ `char`: carattere (forse...)
 - ▶ `float`: numero reale in virgola mobile in singola precisione
 - ▶ `double`: numero reale in virgola mobile in doppia precisione
- Utilizzando questi quattro tipi di dato di base è possibile generare nuovi tipi di dato



Tipi di dato di base

Numero di bit per la rappresentazione

- Il numero di bit con cui viene rappresentato ciascun tipo di base dipende dal **compilatore**
- Esistono di fatto solo un paio di regole “universali”:
 - ▶ Tipo `char`: numero di bit pari alla più piccola unità di memoria indirizzabile e in grado di rappresentare i 52 caratteri maiuscoli e minuscoli, le 10 cifre e altri caratteri di base
 - ▶ Tipo `int`: almeno 16 bit; può quindi certamente rappresentare tutti gli interi compresi fra -2^{15} e $2^{15} - 1$
- `char`: solitamente rappresentato su 8 bit; la codifica dei caratteri dipende dal compilatore (ASCII, UTF8)
- `int`: solitamente rappresentato su 32 bit e consente di rappresentare tutti gli interi compresi fra -2^{31} e $2^{31} - 1$
- `float`: solitamente rappresentato su 32 bit (formato “binary32”, standard IEEE 754)
- `double`: solitamente rappresentato su 64 bit (formato “binary64”, standard IEEE 754)



Variabili

Dichiarazione e inizializzazione

- Una variabile è un dato il cui valore può cambiare durante l'esecuzione del programma
- È un riferimento logico a una locazione di memoria
- **Devono** essere dichiarate (e il loro tipo **deve** essere specificato) prima del loro utilizzo
- **Possono** essere inizializzate durante la dichiarazione
- Dichiarazione:

```
tipo_variabile lista_di_variabili;
```

- Esempi:

```
int ore; /* takes a random value! */  
char key = 'x';  
double pi = 3.14159;  
int minuti, secondi; /* take random values! */
```




Qualificatori

Sintassi

- L'uso di qualificatori può consentire di modificare l'insieme di **numeri rappresentabili** o la **precisione** nella rappresentazione
- Questo è possibile modificando il numero di bit con i quali il dato viene rappresentato
- Sintassi:

```
qualificatore tipo_variabile lista_di_variabili;
```

- Qualificatori: **long**, **short**, **signed**, **unsigned**
- Esempi di dichiarazioni di variabili con qualificatori:

```
long int numero_grande = 2147483648;  
short int numero_piccolo = 321;  
long double pi=3.14159265358979323846264338328;
```



Qualificatori

Uso di qualificatore singolo

- **short int**: consente di rappresentare tutti gli interi compresi almeno fra -2^{15} e $2^{15} - 1$; solitamente 16 bit
- **long int**: consente di rappresentare tutti gli interi compresi almeno fra -2^{31} e $2^{31} - 1$; solitamente 64 bit nel qual caso l'intervallo di valori rappresentabili va da -2^{63} a $2^{63} - 1$
- **signed int**: Equivalente a `int` (nessun effetto)
- **unsigned int**: Stesso numero di bit di `int` ma i numeri interi rappresentati sono senza segno; nel caso 32 bit l'intervallo di valori rappresentabili va da 0 a $2^{32} - 1$
- **signed char**: Equivalente a `char`
- **unsigned char**: Stesso numero di bit di `char` ma senza segno (vedere successivamente)
- **long double**: numero in virgola mobile con precisione non inferiore a quella garantita da `double`; solitamente 128 bit (formato "binary128", standard IEEE 754)

Qualificatori

Uso di qualificatori multipli

- **unsigned short int**: Stesso numero di bit di `short int` ma senza segno; nel caso 16 bit l'intervallo di valori rappresentabili va da 0 a $2^{16} - 1$
- **unsigned long int**: Stesso numero di bit di `long int` ma senza segno; nel caso 64 bit l'intervallo di valori rappresentabili va da 0 a $2^{64} - 1$
- **long long int**: Intero con segno rappresentato su un numero di bit non inferiore a quello del tipo `long int` (spesso equivalente a `long int`)
- **unsigned long long int**: Intero senza segno rappresentato su un numero di bit non inferiore a quello del tipo `unsigned long int` (spesso equivalente a `unsigned long int`)



Costanti

Interi, reali, caratteri e stringhe

- Una **costante** è un dato non modificabile
 - ▶ Uno specifico valore utilizzato nel programma
 - ▶ Un identificativo che gioca il ruolo di tale valore
- Costanti intere (rappresentazione `int`)
 - ▶ forma **decimale**: 1456
 - ▶ forma **ottale**: 0133
 - ▶ forma **esadecimale**: 0x3FF
 - ▶ si può forzare la rappresentazione `long int: 9999999999L`
 - ▶ si può forzare la rappresentazione `unsigned int: 254U`
- Costanti reali
 - ▶ forma **decimale**: 1.2345, -5.4321
 - ▶ forma **esponenziale**: 2.65e-4, -1.46e3
 - ▶ si può forzare la rappresentazione `float: 320.0F`
 - ▶ si può forzare la rappresentazione `long double: 12.3131313131313131L`
- Costanti carattere
 - ▶ Carattere di base: `'x'`; carattere speciale: `'\n'`, `'\t'`, ...
- Costanti stringa (discusse successivamente)

Costanti

Dichiarazione

- Nel caso in cui la costante sia un **identificativo**, essa va dichiarata; è possibile utilizzare il qualificatore `const`

```
const VOTING_AGE = 18;
```

- È anche possibile utilizzare la direttiva `#define`

```
#define VOTING_AGE 18
```

- Nel primo caso il **compilatore** alloca memoria per la costante¹; la costante ha un indirizzo e può essere puntata da un puntatore; la costante è visibile al debugger
- Nel secondo caso il **pre-processor** esegue una sostituzione testuale della stringa "VOTING_AGE" con la stringa "18" in tutto il listato; non c'è allocazione di memoria

¹Alcuni compilatori non allocano memoria se il programma lo consente e se le opzioni di compilazione vengono opportunamente settate

Costanti

Conversioni di formati interi

- Convertire in forma intera la costante esadecimale 6DB

$$(6DB)_{16} = 6 \times 16^2 + 13 \times 16 + 11 = 1755$$

- Eseguiamo la conversione inversa
 - ▶ $1755 \div 16^2 = 6$ con il resto di 219 $= 1755 - 16^2 \cdot 6$
 - ▶ $219 \div 16^1 = 13$ con il resto di 11
 - ▶ $11 \div 16^0 = 11$ con il resto di 0
 - ▶ Quindi: $1755 = 6DB_{16}$

```
int a = 1755;
int b = 0x6DB;
int c;

c=a-b; /* c=0 */
```



Tipi di dato intero a dimensione fissa

Libreria `stdint.h`

- La libreria `stdint.h` (da includere, come al solito, con la direttiva `#include <stdint.h>`) definisce, tra le altre cose, dei tipi di dato interi con dimensioni fisse, indipendentemente dall'architettura
- Tipi di dato interi con segno di dimensioni garantite:
`int8_t`, `int16_t`, `int32_t`, `int64_t`
- Tipi di dato interi senza segno di dimensioni garantite:
`uint8_t`, `uint16_t`, `uint32_t`, `uint64_t`

Outline

- 1 Introduzione
- 2 Variabili e costanti
- 3 Input/output di base**
- 4 Rappresentazione binaria dei dati
- 5 Operatori
- 6 Conversioni di tipo
- 7 Array e puntatori
- 8 Funzioni
- 9 Controllo del flusso
- 10 Input/output da file
- 11 Variabili aleatorie in C
- 12 Numeri complessi in C
- 13 Elaborazione in virgola fissa

Libreria per l'I/O standard

- Il linguaggio C definisce una libreria standard di funzioni per la gestione di dell'input/output di base
- Mediante la direttiva `#include <stdio.h>` è possibile comunicare al compilatore l'intenzione di usare tali funzioni
 - ▶ Tale direttiva viene solitamente inclusa all'inizio del file sorgente
- Il file `stdio.h` include tutte le informazioni necessarie al compilatore per accedere alle funzioni per l'input/output standard
- Quando un programma C viene eseguito, tre file vengono aperti automaticamente: lo **standard input**, lo **standard output** e lo **standard error**
- La libreria per l'I/O standard consente di leggere dati dallo standard input (default: tastiera), scrivere sullo standard output (default: monitor), inviare messaggi di errore sullo standard error (default: monitor)



Funzione printf

Sintassi, esempi

- Scrive dati sullo standard output e ha la seguente sintassi:

```
printf(format_string, arg1, arg2, ...)
```

- `format_string` è una stringa che contiene sia **testo ordinario** che uno **specificatore di formato** per ciascun argomento `arg1, arg2, ...`

```
const double pi = 3.1415926535898;  
int a=2, b=3;  
printf("Hello World!\n"); /* prints Hello World! */  
printf("Pi=%lf\n", pi); /* prints Pi=3.141593 */  
printf("Pi=%.13lf\n", pi);  
                /* prints Pi=3.1415926535898 */  
printf("The sum of %d and %d is %d.\n", a, b, a+b);  
/* prints The sum of 2 and 3 is 5 */
```

Funzione scanf

Sintassi, esempi

- Legge dati dallo standard input e ha la seguente sintassi:

```
scanf(format_string, arg1, arg2, ...)
```

- Consente di leggere più dati contemporaneamente
- Anche in questo caso la `format_string` contiene **testo ordinario** e **specificatori di formato**
- Nel caso di ingressi multipli, ignora la pressione di spazi e tab, e la pressione di return intermedi

```
int day, month, year;
int a, b, c;
scanf("%d/%d/%d", &day, &month, &year);
/* The input MUST contain three integers separated by
   slashes */
scanf("%d %d %d", &a, &b, &c);
/* 34 56
45 produces the desired effect regardless of blanks,
   tabs, intermediate returns */
```



Funzioni `getchar` e `putchar`

Sintassi, esempi

- La libreria standard definisce le due funzioni, `putchar` e `getchar`
- `putchar` scrive un **singolo carattere** su standard output

```
char key = 'a';  
putchar(key);
```

- `getchar` legge un **singolo carattere** da standard input

```
char key;  
key = getchar();
```

- Per la lettura e scrittura di singoli caratteri queste funzioni sono più efficienti di `printf` e `scanf`

Outline

- 1 Introduzione
- 2 Variabili e costanti
- 3 Input/output di base
- 4 Rappresentazione binaria dei dati**
- 5 Operatori
- 6 Conversioni di tipo
- 7 Array e puntatori
- 8 Funzioni
- 9 Controllo del flusso
- 10 Input/output da file
- 11 Variabili aleatorie in C
- 12 Numeri complessi in C
- 13 Elaborazione in virgola fissa

Rappresentazione binaria di interi

Interi con segno: rappresentazione in **complemento a 2**

```
short int x=3674,y=-21897,z;
/*
Binary word in x is 0000111001011010
Binary word in y is 1010101001110111

Obtaining the binary representation of y:
Express 21897:      0101010110001001
Complement all bits: 1010101001110110
Sum 1:              1010101001110110+
                                   1=
                        -----
                        1010101001110111
*/
```

- Il bit più significativo (a sinistra) indica il segno (0: positivo; 1: negativo)
- Ripetendo l'operazione sul numero in complemento a 2 si ottiene il modulo del numero.... provare!

Rappresentazione binaria di interi

Interi senza segno

- Per i valori interi dichiarati `unsigned` viene utilizzata la **codifica naturale**
 - ▶ Esempio (interi da 0 a 3): $0 \mapsto 00$; $1 \mapsto 01$; $2 \mapsto 10$; $3 \mapsto 11$
- Una variabile intera con segno e una intera senza segno possono avere valori diversi ma contenere la medesima parola binaria

```
short int x = -9;
unsigned short int y = 65527;

/*
Binary word in x is 111111111110111
Binary word in y is 111111111110111
*/
```

Rappresentazione binaria di interi

Somma algebrica fra interi con segno

- In complemento a 2, l'operazione di sottrazione può essere eseguita come somma algebrica

```
short int x=25,y=107;
z=x-y;
/*
Binary representation of   25: 0000000000011001
Binary representation of  107: 0000000001101011
Binary representation of -107: 1111111110010101

Calculate z=x+(-y):
0000000000011001+
1111111110010101=
-----
1111111110101110
This yields z=-82 as expected
*/
```


Rappresentazione binaria di caratteri

Caratteri come interi

- Piuttosto che come un carattere, è conveniente pensare al valore di una variabile `char` come una **parola binaria** (tipicamente di lunghezza 8 bit)
- Tale parola binaria può essere specificata mediante un **numero intero**
 - ▶ Il tipo `char` è equivalente a `signed char`, quindi l'intero può essere con segno (interi rappresentati: da -128 a 127)
 - ▶ Mediante qualificatore (`unsigned char`) è possibile rendere l'intero senza segno (interi rappresentati: da 0 a 255)

```
char c1=40,c2=-98;  
char c3=137;  
/* actual value of c3 is -119 (no compiler warning) */  
char c4=290;  
/* actual value of c4 is 34 (with compiler warning) */  
unsigned char c5=137; /* actual value of c5 is 137 */
```

Rappresentazione binaria di caratteri

Codifica ASCII

- L'associazione fra parole binarie e caratteri del set rappresentato è in linea di principio arbitraria
- Una codifica molto comune è la codifica **ASCII**
 - Utilizza 7 bit con il bit più significativo a 0

0 nul	16 dle	32 sp	48 0	64 @	80 P	96 `	112 p
1 soh	17 dc1	33 !	49 1	65 A	81 Q	97 a	113 q
2 stx	18 dc2	34 "	50 2	66 B	82 R	98 b	114 r
3 etx	19 dc3	35 #	51 3	67 C	83 S	99 c	115 s
4 eot	20 dc4	36 \$	52 4	68 D	84 T	100 d	116 t
5 enq	21 nak	37 %	53 5	69 E	85 U	101 e	117 u
6 ack	22 syn	38 &	54 6	70 F	86 V	102 f	118 v
7 bel	23 etb	39 '	55 7	71 G	87 W	103 g	119 w
8 bs	24 can	40 (	56 8	72 H	88 X	104 h	120 x
9 ht	25 em	41)	57 9	73 I	89 Y	105 i	121 y
10 nl	26 sub	42 *	58 :	74 J	90 Z	106 j	122 z
11 vt	27 esc	43 +	59 ;	75 K	91 [	107 k	123 {
12 np	28 fs	44 ,	60 <	76 L	92 \	108 l	124
13 cr	29 gs	45 -	61 =	77 M	93]	109 m	125 }
14 so	30 rs	46 .	62 >	78 N	94 ^	110 n	126 ~
15 si	31 us	47 /	63 ?	79 O	95 _	111 o	127 del

Rappresentazione binaria di caratteri

Associazione tra parola binaria e carattere

- L'associazione fra parola binaria e carattere diviene visibile, ad esempio, quando:
 - ▶ a una variabile `char` (o `unsigned char`) viene assegnato un carattere come valore
 - ▶ si chiede di stampare (come carattere) il valore della variabile
- Le parole binarie che rappresentano numeri negativi (o numeri interi positivi maggiori di 127, nel caso `unsigned`) non sono associate ad alcun carattere

```
char c1=40,c2=-98;
printf("c1=%c\n",c1); /* prints c1=( */
printf("c2=%c\n",c2); /* might print c2=? */

char c3=290;
printf("c3=%c\n",c3); /* prints c3=" (compiler warn)
*/

char c4='(';
printf("c4=%d\n",c4); /* prints c1=40 +/
```



Stringhe

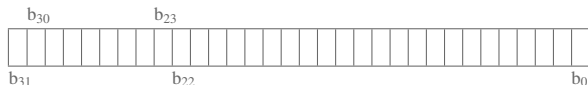
- Una costante stringa è una sequenza di caratteri (in numero maggiore o uguale a zero) racchiusa fra virgolette
 - ▶ “Hello World!”, “Io sono una stringa”, “x”, “”
- In C **non esiste** il tipo di dato stringa, ma una costante stringa può essere assegnata a un **array di caratteri**
- Internamente il compilatore appende a ciascuna stringa il carattere speciale ‘\0’ per marcare la fine della stringa
- Occupazione fisica di memoria di una stringa con n caratteri: $n + 1$ moltiplicato per il numero di bit per carattere

```
char my_string[16] = "I am a string";  
/* my_string[0]='I'; my_string[15]='\nul' */  
printf("%hhd\n",my_string[0]); /* prints 73 */  
printf("%hhd\n",my_string[15]); /* prints 0 */  
my_string[9] = 'z'; /* no compiler error */  
my_string = "hello"; /* compiler error */
```

Rappresentazione binaria di reali

Reali in virgola mobile in singola precisione

- Il formato “binary32”, tipicamente utilizzato nel caso `float`, prevede una rappresentazione a 32 bit dei quali:
 - 1 bit per il **segno** (b_{31}): $0 \rightarrow +$, $1 \rightarrow -$
 - 8 bit per l'**esponente** ($b_{30} - b_{23}$)
 - 23 bit per la **mantissa** ($b_{22} - b_0$)



- Esponente**: codificato come un intero “unsigned” a 8 bit
- Mantissa**: 23 bit memorizzati; il bit più significativo è implicito, quindi la precisione è di 24 bit
- Il numero reale f rappresentato è dato dalla relazione

$$f = (-1)^{b_{31}} \times (1.b_{22} \dots b_0)_2 \times 2^{(b_{30} \dots b_{23})_2 - 127}$$

$$= (-1)^{b_{31}} \times \left(1 + \sum_{i=1}^{23} b_{23-i} 2^{-i}\right) \times 2^{(\sum_{i=0}^7 b_{23+i} 2^i) - 127}$$



Rappresentazione binaria di reali

Conversione da decimale a binary32

- Si convertono in binario, separatamente, le parti intera e decimale
- Si moltiplica per 2^0 e si sposta la virgola (“virgola mobile”) per avere un solo bit 1 prima di essa, aggiustando l'esponente
- Si somma l'offset 127 all'esponente e si aggiunge il bit di segno
- Esempio: $f = -336.3125$
 - ▶ $336 = (101010000)_2$
 - ▶ $0.3125 = (0.0101)_2$
 - ▶ $336.3125 = (101010000.0101)_2$
 - ▶ $336.3125 = (101010000.0101)_2 \times 2^0$
 - ▶ $336.3125 = (1.010100000101)_2 \times 2^8$
 - ▶ Mantissa: 0101000001010000000000
 - ▶ Esponente: $8+127=135$
 - ▶ Esponente: 1000111
 - ▶ Bit di segno: 1
 - ▶ $f \rightarrow 110001110101000001010000000000$

Rappresentazione binaria di reali

Conversione da binary32 a decimale

- Consideriamo la rappresentazione “binary32” di $f = 0.4376$
- Risulta (seguendo la precedente procedura)

$f \rightarrow$ 0011111011100000000110100011011

- Verifica:
 - ▶ Gli 8 bit “unsigned” dell’esponente (01111101) codificano il numero intero 125
 - ▶ I 23 bit della mantissa (1100000000110100011011), assieme al bit implicito, codificano il numero

$$(1.1100000000110100011011)_2 = 1.7504$$

- ▶ Otteniamo

$$\begin{aligned} f &= (-1)^0 \times 1.7504 \times 2^{125-127} \\ &= 1.7504 \times 2^{-2} \\ &= 0.4376 \end{aligned}$$

Rappresentazione binaria di reali

Valori interi rappresentabili e precisione

- I due valori `0x00` e `0xFF` dell'esponente sono **riservati**; il massimo valore dell'esponente è dunque $254 - 127 = 127$
- Il modulo del massimo valore rappresentabile è l'intero $(1 + \sum_{i=1}^{23} 2^{-i})2^{127} \approx 3.402823466385288598 \cdot 10^{38}$
- Per confronto, il massimo intero rappresentabile in complemento a 2 su 32 bit è $2.147483647 \cdot 10^9$
- Qualunque intero n : $-2^{24} \leq n \leq 2^{24}$ viene rappresentato senza arrotondamento
- Per ogni $24 \leq i \leq 128$, ciascun intero n : $2^i + 1 \leq |n| \leq 2^{i+1}$ viene arrotondato a un multiplo di 2^{i-23}

```
float f=2267650600228229401496703205377.0;
/* 2^100+1 <= f <= 2^101, i=100 */
printf("%f",f);
/* prints 2267650615275695621373392060416.000000\\
2267650615275695621373392060416/2e77=15006053 */
```


Rappresentazione binaria di reali

Decimali e precisione

- Il formato “binary32” ha una **precisione 6–9** sulle cifre decimali dopo la virgola
- Conversione formato decimale → formato “binary32”: reversibile se il numero di cifre dopo la virgola è al più 6
- Conversione formato “binary32” → formato decimale: reversibile se, nel formato decimale, il numero di cifre dopo la virgola è al più 9

```
float f=3.992367,g=3.9923677567567;  
printf("%.6f",f); /* prints 3.992367 */  
printf("%.13f",g); /* prints 3.9923677444458 */
```

Rappresentazione binaria di reali

Reali in virgola mobile in doppia precisione

- I formati “binary64” e “binary128” sono simili al formato “binary32” precedentemente descritto
 - ▶ Formato “binary64”: 1 bit per il **segno**, 11 bit per l'**esponente**, 52 bit per la **mantissa**; l'offset all'esponente è pari a 1023
 - ▶ Formato “binary128”: 1 bit per il **segno**, 15 bit per l'**esponente**, 112 bit per la **mantissa**; l'offset all'esponente è pari a 16383
- Aumentano la precisione (oltre ad estendere il range di valori rappresentabili)
 - ▶ “binary64”: precisione 13–15 (contro 6–9)
 - ▶ “binary128”: precisione 33–36 (contro 13–15)

```
double g=3.9923677567567;  
printf("%.13lf",g); /* prints 3.9923677567567 */
```

Outline

- 1 Introduzione
- 2 Variabili e costanti
- 3 Input/output di base
- 4 Rappresentazione binaria dei dati
- 5 Operatori**
- 6 Conversioni di tipo
- 7 Array e puntatori
- 8 Funzioni
- 9 Controllo del flusso
- 10 Input/output da file
- 11 Variabili aleatorie in C
- 12 Numeri complessi in C
- 13 Elaborazione in virgola fissa

Operatori aritmetici

- Cinque operatori aritmetici binari: “+” (addizione), “-” (sottrazione), “*” (moltiplicazione), “/” (divisione), “%” (modulo)
- Un operatore aritmetico unario: “-”
- L'espressione a/b restituisce il quoziente della divisione se a e b sono interi
- L'espressione $a\%b$ restituisce il resto della divisione se a e b sono interi

```
int a=15, b=7, q_int, r_int;
float c=4.376e+01, d=1.9, q_float;
q_int=a/b;    /*q_int=2*/
r_int=a%b;    /* r_int=1 */
q_float=c/d;  /* q_float=2.303158e+01 */
c=-q_float    /* c=-2.303158e+01 */
```

N.B: Per l'elevamento a potenza c'è la funzione `pow(a, exp)` nella libreria `math.h` (e tante altre funzioni più complesse)

Operatori di incremento e decremento

- Due operatori unari che consentono di incrementare e decrementare le variabili:
 - ▶ L'operatore ++ somma il valore 1 all'operando
 - ▶ L'operatore -- sottrae il valore 1 all'operando
- Ciascuno dei due operatori può essere usato **prima** o **dopo** l'operando
 - ▶ Quando l'operatore segue l'operando, questo viene incrementato di 1 **al termine** dell'istruzione
 - ▶ Quando l'operatore precede l'operando, questo viene incrementato di 1 **all'inizio** dell'istruzione

```
int x,y=3;  
x=y++; /* x=3, y=4 */
```

```
int x,y=3;  
x=++y; /* x=4, y=4 */
```

Operatori di assegnamento

- L'operatore di assegnamento **semplice** "=" consente di assegnare uno specifico valore a una variabile

```
var = expr;
```

- L'espressione `expr` viene dapprima valutata e il risultato diviene il valore della variabile `var`
- Operatori di assegnamento **composti**: consentono di scrivere in forma compatta un'istruzione del tipo `var = var op expr`, dove `op` è un operatore aritmetico binario

```
var += expr; /* var = var + expr; */  
var -= expr; /* var = var - expr; */  
var *= expr; /* var = var * expr; */  
var /= expr; /* var = var / expr; */  
var %= expr; /* var = var % expr; */
```

```
int x=2,y=3;  
x *= y+1; /* x = x*(y+1) = 8*/
```

Ordine di valutazione delle espressioni

- Operatori aritmetici: usuali regole di precedenza (associatività a sinistra nel caso di stessa precedenza)
- Le parentesi “ () ” forzano l'ordine di valutazione
- Gli operatori aritmetici hanno la precedenza sugli operatori di assegnamento
- L'uso delle parentesi, anche ridondante, è sempre consigliabile

```
int x;  
x = 4+3*7;           /* x=25 */  
x = (4+3)*7;         /* x=49 */  
x = 4*3+4*3+2;       /* x=26 */  
x = 4*( (3+4)*(3+2) ); /* x=140 */  
x = 4/(3*3);          /* x=0 */  
x = (4/3)*3;          /* x=3 */  
  
int x=2;  
x += 4%3*3;           /* x=5 */
```



Operatori bitwise

Operatori disponibili

- Il linguaggio C offre la possibilità di manipolare i dati a livello di **singolo bit**
 - ▶ Tale funzionalità è molto utile per una programmazione a basso livello (ad esempio nei **sistemi embedded**)
- Esistono sei operatori bitwise

Operatore	Descrizione
&	AND bit a bit
	OR bit a bit
^	XOR bit a bit
~	NOT su ciascun bit
>>	Shift verso destra
<<	Shift verso sinistra

- Gli operatori bitwise richiedono operandi **interi**, ma non necessariamente dello stesso tipo

Operatori bitwise

Composizione con operatori di assegnamento

- Gli operatori di assegnamento **composti** possono essere costruiti anche con operatori bitwise
- Consentono di scrivere in forma compatta istruzioni di assegnamento che includono, sulla destra dell'operatore di assegnamento, operatori bitwise

```
char c_1='a';           /* 01100001 */
char c_2='b';           /* 01100010 */
char c_3='c';           /* 01100011 */

c_3&=c_1;               /* 01100001 */
printf("%c\n", c_3);    /* prints a */

c_3|=c_2;               /* 01100011 */
printf("%c\n", c_3);    /* prints c */

c_3>>=1;                /* 00110001 */
printf("%c\n", c_3);    /* prints 1 */
```

Operatori bitwise

Tabelle della verità (AND, OR, XOR, NOT)

AND

Op1	Op2	Ris
0	0	0
0	1	0
1	0	0
1	1	1

OR

Op1	Op2	Ris
0	0	0
0	1	1
1	0	1
1	1	1

XOR

Op1	Op2	Ris
0	0	0
0	1	1
1	0	1
1	1	0

NOT

Op	Ris
0	1
1	0



Operatori bitwise

Esempi (AND, OR, XOR, NOT)

```
short int a=28200; /* 0110111000101000 */
unsigned short int b=5467; /* 0001010101011011 */

printf("%hd", a^0x4B0);
/* (4B0)_16 = (0000010010110000)_2
the bit-wise XOR returns 0110101010011000
the instruction prints 27288 */

printf("%hd", a&b);
/* the bit-wise AND returns 0000010000001000
the instruction prints 1032 */

printf("%hd", a|0x4B0);
/* (-4B0)_16 = (1111101101010000)_2
the bit-wise OR returns 1111111101111000
the instruction prints -136 */

printf("%hu", ~b);
/* the NOT returns 1110101010100100
the instruction prints 60068 (possible compiler warning) */

printf("%hd", ~b);
/* the NOT returns 1110101010100100
the instruction prints -5468 (possible compiler warning) */
```

Operatori bitwise

Esempi (AND, OR, XOR, NOT)

- Nel caso in cui gli operandi siano rappresentati con un diverso numero di bit, l'operando più corto viene esteso
- Se l'operando che viene esteso è "signed", l'estensione include il segno

```
char c_neg=-30;      /* 11100010 */
char c_pos=30;       /* 00011110 */
short int a = 0xFFFF; /* 1111111111111111 */
short int x;

x=c_neg&a;
printf("x=%hd", x);
/* c_neg is extended to 1111111111100010 before & operation
the bit-wise AND returns 1111111111100010
the instruction prints x=-30 */

x=c_pos&a;
printf("x=%hd", x);
/* c_pos is extended to 0000000000011110 before & operation
the bit-wise AND returns 0000000000011110
the instruction prints 30 */
```

Operatori bitwise

Operatori di shift

- Consentono di specificare il numero di volte in cui i bit della parola binaria devono essere traslati
- Operatore `<<` : introduce un numero di bit 0 a destra pari al numero di traslazioni
- Operatore `>>` : introduce un numero di bit a sinistra pari al numero di traslazioni. Spesso (dipende dal compilatore):
 - ▶ Intero "signed": viene esteso il segno (**shift aritmetico**)
 - ▶ Intero "unsigned": vengono introdotti bit 0 (**shift logico**)

```
char c_s=-126; unsigned char c_u=130; /* 10000010 for both */

c_s>>=1; /* signed: results in 11000001 */
printf("%hhd\n",c_s); /* prints -63 */

c_u>>=1; /* unsigned: results in 01000001 */
printf("%hhu\n",c_u); /* prints 65 */

c_s<<=6; c_u<<=6; /* 10000000 for both */
printf("%hhd\n",c_s); /* prints -64 */
printf("%hhu\n",c_u); /* prints 64 */
```

Operatori bitwise

Shift a sinistra e moltiplicazione

- Sotto certe condizioni, se a è una variabile di tipo intero l'operazione di shift a sinistra ($<<$) di k posizioni restituisce il **prodotto** del valore di a per 2^k
- La variabile a può essere “signed” o “unsigned”
- L'equivalenza tra shift e prodotto non è tuttavia più valida quando si verifica una condizione di **overflow**

```
unsigned char c_u;  char c_s;

c_u=23; c_u<<=3; /* 00010111 -> 10111000 */
printf("%hhu\n",c_u); /* prints 184=23x2^3 */

c_u=23; c_u<<=4; /* 00010111 -> 01110000 */
printf("%hhu\n",c_u); /* prints 112 (overflow) */

c_s=-23; c_s<<=2; /* 11101001 -> 10100100 */
printf("%hhd\n",c_s); /* prints -92=-23x2^2 */

c_s=-23; c_s<<=3; /* 11101001 -> 01001000 */
printf("%hhd\n",c_s); /* prints 72 (overflow) */
```

Operatori bitwise

Shift a destra e divisione

- Sotto certe condizioni, se a è una variabile di tipo intero l'operazione di shift a destra ($>>$) di k posizioni restituisce il **quoziente** della divisione del valore di a per 2^k
- La variabile a può essere “signed” o “unsigned”
- Nel caso di interi negativi dispari il risultato è pari al quoziente meno 1

```
unsigned char c_u; char c_s;

c_u=23; c_u>>=1; /* 00010111 -> 00001011 */
printf("%hhu\n",c_u); /* prints 11 (23/2=11) */

c_u=23; c_u>>=3; /* 00010111 -> 00000010 */
printf("%hhu\n",c_u); /* prints 2 (23/8=2) */

c_s=-23; c_s>>=1; /* 11101001 -> 11110100 */
printf("%hhd\n",c_s); /* prints -12 (-23/2=-11) */

c_s=-77; c_s>>=3; /* 10110011 -> 11110110 */
printf("%hhd\n",c_s); /* prints -10 (-77/8=-9) */
```

Outline

- 1 Introduzione
- 2 Variabili e costanti
- 3 Input/output di base
- 4 Rappresentazione binaria dei dati
- 5 Operatori
- 6 Conversioni di tipo**
- 7 Array e puntatori
- 8 Funzioni
- 9 Controllo del flusso
- 10 Input/output da file
- 11 Variabili aleatorie in C
- 12 Numeri complessi in C
- 13 Elaborazione in virgola fissa

Promozione implicita

- Qualunque costante o variabile `char` o `short` (signed o unsigned) viene sempre “promossa” dal compilatore a `int` (signed o unsigned) prima di eseguire qualunque operazione che la contiene come operando
- Questa promozione implicita, tuttavia, **non** modifica la dimensione di una variabile o costante

```
char key;  
int n;  
  
n=sizeof('A'); /* n=4 (surprisingly!) */  
key='A';  
n=sizeof(key); /* n=1 (not surprisingly) */  
  
key=key-3; /* during evaluation key promoted to int */  
n=sizeof(key); /* n=1 (not surprisingly) */
```

Conversione di tipo implicita

Algoritmo di conversione

- Dopo la promozione implicita, se gli operandi sono dello stesso tipo l'operazione viene eseguita
- Altrimenti, se uno degli operandi è **reale**:
 - ▶ Se `long double`: l'altro operando convertito a `long double`
 - ▶ Altrimenti, se `double`: l'altro operando convertito a `double`
 - ▶ Altrimenti, se `float`: l'altro operando convertito a `float`
- Altrimenti (operandi **interi**):
 - ▶ Se un operando è `unsigned long`: l'altro operando convertito a `unsigned long`
 - ▶ Altrimenti, se `long`, l'altro operando viene convertito a `long2`
 - ▶ Altrimenti, se `unsigned int`: l'altro operando convertito a `unsigned int`
 - ▶ Altrimenti, l'operazione viene eseguita con operandi `int`

²Eccezione: se l'altro operando è `unsigned int` e `sizeof(long) == sizeof(unsigned int)`, entrambi gli operandi convertiti a `unsigned long`

Conversione di tipo implicita

Esempi

```
int i=-1; unsigned int u=2;
if (i<u) printf("i is smaller than u\n");
else printf("u is smaller than i\n");

/* prints: u is smaller than i (!) */
```

```
float f=50/13*14/5.0;

/* f=8.4 because 50/13 evaluates to 3 (int operands), then
3*14 evaluates to 42 (int operands), then 42 is converted
to double (binary64) and 42/5.0 evaluates to 8.4, then
8.4 is converted to float (binary32) */
```

```
float f=50/13.0*14/5;

/* f=10.769231 because 50 is converted to double and 50/13.0
evaluates to 3.846153, then 14 is converted to double
(binary64) and 3.846153*14 evaluates to 53.846154, then
5 is converted to double (binary64) and 53.846154/5
evaluates to 10.769231, then 10.769231 is converted to
float (binary32) */
```

Cast

- Il cast è un'operazione che consente di **forzare** il tipo di dato (cioè la rappresentazione interna) di una costante, variabile o espressione
- Un cast viene eseguito ponendo fra parentesi, davanti alla costante, variabile o espressione un tipo di dato

```
float f=(float) (50/13*14/5);  
/* f=8.0 as cast is executed at the very end and  
has no effect */  
  
float f=(50/13*14/(float)5);      /* f=8.4 */  
  
f=((int)8.5*2)/(double)7;  
/* f=2.285714 as: (int)8.5*2 evaluates to 16  
(2's complement), then 7 is converted to double  
(binary64) by cast, then 16 is converted to  
double (binary64) by implicit conversion, then  
16/7 evaluates to 2.285714 (binary64), finally  
2.285714 is converted to binary32 representation */
```

Outline

- 1 Introduzione
- 2 Variabili e costanti
- 3 Input/output di base
- 4 Rappresentazione binaria dei dati
- 5 Operatori
- 6 Conversioni di tipo
- 7 Array e puntatori**
- 8 Funzioni
- 9 Controllo del flusso
- 10 Input/output da file
- 11 Variabili aleatorie in C
- 12 Numeri complessi in C
- 13 Elaborazione in virgola fissa

Array monodimensionale

Dichiarazione, accesso elementi

- Un array è un gruppo di dati dello stesso tipo
- Viene assegnato un nome all'intero gruppo; il generico dato è accessibile mediante un indice fra parentesi quadre
- Dichiarazione:

```
specificatore_di_tipo nome_array[numero_elementi];
```

- È bene che il parametro `numero_elementi` sia una **costante** intera positiva

```
#define m 7
const int n=10;
int a[5],b[n];
int c[m]={0,0,1,1,0,0,1}; /* initialization */
a[0]=3; b[9]=-5;
c[9]=0; /* possible compiler warning but not error */
```

- Il nome dell'array rappresenta l'**indirizzo** del primo elemento dell'array ed è una **costante**

Array multidimensionale

Dichiarazione, accesso elementi

- È possibile dichiarare array multidimensionali
- Il caso bidimensionale è utile, ad esempio, per definire una **matrice**
- Dichiarazione (esempio bidimensionale):

```
specificatore_di_tipo nome_array  
    [numero_elementi_1][numero_elementi_2];  
}}
```

```
#define n_rows 2  
#define n_cols 3  
int a[n_rows][n_cols] = {{1,1,0}, {0,1,1}};  
  
/* a =  1 1 0  
        0 1 1 */  
a[1][2] += 3;  
/* a =  1 1 0  
        0 1 4 */
```



Puntatori

Dichiarazione, dereferenziazione

- Puntatore: variabile il cui valore è l'indirizzo di memoria di un'altra variabile
- Dichiarazione:

```
specificatore_di_tipo *nome_puntatore;
```

- È possibile assegnare a un puntatore l'indirizzo di una variabile utilizzando l'operatore unario **&**
- È possibile accedere mediante il puntatore al valore della variabile puntata utilizzando l'operatore unario ***** (operatore di "dereferenziazione")

```
int v=3;      /* v: an integer variable */
int *ptr;     /* ptr: a pointer to a memory address
               storing an integer */

ptr=&v;        /* ptr "points" v */
*ptr++;       /* v=4 */
```


Dimensione di un puntatore

Puntatore a `void`

- In C è possibile definire un puntatore a `void` (`void *ptr;`)
- Si tratta di un puntatore a una locazione di memoria per la quale **non** viene specificato il tipo di dato memorizzato
- Può puntare dati di **qualsunque** tipo (`int`, `float`, `char`, ...)
- Non ammette l'uso dell'operatore di dereferenziazione a meno di **cast** esplicito

```
void *v_ptr;
int *i_ptr;
int a=10,b;

i_ptr=&a;
v_ptr=i_ptr;           /* equivalent to v_ptr=&a; */
b=*v_ptr;              /* compiler error */
b=* (int*)v_ptr;       /* b=10 */
```

- È molto utile se si vuole passare a una funzione un argomento il cui tipo non è noto a priori

Dimensione di un puntatore

Considerazioni

- Il numero di bit con cui un puntatore viene rappresentato dipende dall'**architettura** e dal **modello di memoria**
- Lo standard C non impone che puntatori a tipi di dato diversi siano rappresentati sullo stesso numero di bit
 - ▶ Esempio: in un'architettura con `int` a 4 byte allineati, un puntatore a `char` potrebbe richiedere 2 bit in più rispetto a un puntatore a `int`
- Stabilisce però che il cast di un qualunque puntatore a `void` deve avvenire senza perdita di informazione
- Spesso la dimensione dei puntatori è fissa e pari a quella di un puntatore a `void`: 8 byte su macchine a 64 bit, 4 byte su macchine a 32 bit

```
int *ptr;
printf("%lu\n", sizeof(ptr));
/* may print 8 by default but 4 under a -m32 compiler
option */
```

Puntatori e array

Puntatori come array

- Consideriamo la seguente dichiarazione:

```
int v[7]; int *ptr;
```

- Possiamo far puntare `ptr` al primo elemento dell'array

```
ptr=&v[0]; /* equivalent: ptr=v; */
```

- A questo punto possiamo usare il puntatore per accedere agli elementi dell'array

```
int n;  
n=*ptr; /* n=v[0] */  
n=*(ptr+3); /* n=v[3] */
```

- In `ptr+3`, 3 rappresenta un **offset**; poiché `ptr` è un puntatore a intero, tale offset è pari a `3*sizeof(int)` byte
- Possiamo adottare per `ptr` la **sintassi tipica degli array**

```
n=ptr[1]; /* n=v[1] */}}
```

Puntatori e array

Array come puntatori

- Così come è possibile adottare la sintassi degli array per i puntatori, possiamo usare la sintassi dei puntatori con gli array

```
float x[2] = {0.1, 0.0};  
*(x+1) = -0.4; /* equivalent: x[1]=-0.4; */
```

- Le due sintassi sono equivalenti e intercambiabili
- Esiste tuttavia una **differenza sostanziale** fra un array e un puntatore: il nome di un array è l'indirizzo **costante** (non modificabile) del suo primo elemento; un puntatore è invece una **variabile**

```
int v[3]={2,-1,5}, *ptr;  
ptr=v;  
v=ptr; /* compiler error */  
ptr+=1;  
v+=1; /* compiler error */
```

Puntatori e array

Esempi

```
int a=-1, b=2; int *ptr;

ptr=&a;      /* ptr "points" to a */
b=*ptr;     /* b=-1 */
ptr[0]=0;   /* a=0 */
ptr=&b;      /* ptr "points" to b */
*ptr+=3;    /* b=2 */
```

```
int v[3]; int *ptr;

printf("%p\n", (void*)&v[0]); /* may print 0x7fff5fb88b2c */
printf("%p\n", (void*)&v[1]); /* prints 0x7fff5fb88b30 */
printf("%p\n", (void*)&v[2]); /* prints 0x7fff5fb88b34 */
ptr=&v[0];
printf("%p\n", (void*)ptr); /* prints 0x7fff5fb88b2c */
ptr+=1;
printf("%p\n", (void*)ptr); /* prints 0x7fff5fb88b30 */
ptr+=1;
printf("%p\n", (void*)ptr); /* prints 0x7fff5fb88b34 */
```

Allocazione dinamica di memoria

Funzione `malloc`

- Utilizzata per allocare memoria durante l'esecuzione di un programma; il prototipo si trova nella libreria `stdlib.h`
- Prototipo:

```
void *malloc(size_t number);
```

- Restituisce un **puntatore a void** che punta la prima cella (tipicamente il primo byte) del blocco di memoria allocato
- L'argomento **number** rappresenta il numero di celle di memoria (tipicamente il numero di byte) da allocare
- **size_t** è un tipo di dato speciale che viene utilizzato per la dimensione di variabili; di fatto è un intero senza segno rappresentato su almeno 16 bit

Allocazione dinamica di memoria

Cast esplicito, funzione `free`

- `malloc` restituisce un puntatore a `void` poiché il tipo di dato per il quale essa deve allocare memoria non è noto a priori
- Solitamente essa viene chiamata con un **cast** esplicito
- Per de-allocare dinamicamente la memoria si utilizza la funzione `free`. Prototipo:

```
void free(void *);
```

```
char *c_ptr; int *i_ptr; double *d_ptr;

c_ptr=(char*)malloc(10*sizeof(char));
/* allocates 10 memory cells of 1 byte each */

i_ptr=(int*)malloc(10*sizeof(int));
/* allocates 10 memory cells of 4 bytes each */

d_ptr=(double*)malloc(10*sizeof(double));
/* allocates 10 memory cells of 8 bytes each */

free(c_ptr); free(i_ptr); free (d_ptr);
```

Outline

- 1 Introduzione
- 2 Variabili e costanti
- 3 Input/output di base
- 4 Rappresentazione binaria dei dati
- 5 Operatori
- 6 Conversioni di tipo
- 7 Array e puntatori
- 8 Funzioni**
- 9 Controllo del flusso
- 10 Input/output da file
- 11 Variabili aleatorie in C
- 12 Numeri complessi in C
- 13 Elaborazione in virgola fissa

Funzioni

Utilità delle funzioni

- Un programma C consiste in una o più funzioni
- Esso deve contenere almeno una funzione chiamata `main`
- L'esecuzione del programma inizia con la prima istruzione del `main`
- Le funzioni sono utili per **semplificare** la scrittura di codice per realizzare obiettivi complessi
- L'obiettivo viene suddiviso in obiettivi intermedi più semplici, per ognuno dei quali viene implementata una funzione specifica
- **Leggibilità**: il codice risulta più compatto e leggibile
- **Riusabilità**: una funzione può essere utilizzata più volte all'interno del medesimo progetto, oppure in progetti diversi



Funzioni

Definizione di una funzione

- In C, le funzioni hanno la seguente struttura generale (**definizione** della funzione):

```
tipo_risultato nome_funzione(lista_argomenti)
{
    dichiarazioni
    tab istruzioni
}
```

- Una funzione può invocare un'altra funzione passando ad essa dei dati mediante la `lista_argomenti` della funzione che viene invocata
- La funzione che è stata invocata può restituire un valore di ritorno, il cui tipo è `tipo_risultato`, alla funzione dalla quale è stata invocata

Funzioni

Argomenti di una funzione

- `lista_argomenti`: specifica il **tipo dei dati** che la funzione si aspetta di ricevere, e assegna ad essi degli **identificatori**

```
void square(double x)
{
    printf("Square of %lf is %lf", x, x*x);
    return;
}
```

- La funzione si aspetta di ricevere un numero reale in doppia precisione quando viene chiamata
- Nel corpo della funzione, questo argomento è chiamato `x`
- Essa non restituisce alcun valore (`void`)
- Esempio di invocazione:

```
double z = 3.17;

square(z); /* prints: Square of 3.17 is 10.0489 */
```



Funzioni

Valore restituito da una funzione

- Una funzione può restituire **uno ed un solo** valore; il valore restituito è di tipo `tipo_risultato`
- Ad esempio, possiamo modificare la precedente funzione `square` in questo modo:

```
double square(double x)
{
    return (x*x);
}
```

- Così facendo, la funzione `square` restituisce un valore di tipo `double` (rappresentazione “binary64”)
- L’istruzione `return` informa il compilatore circa il valore che vogliamo restituire

```
double cateto_1, cateto_2, quadrato_ipotenusa;

quadrato_ipotenusa=square(cateto_1)+square(cateto_2);
```

Funzioni

Prototipo di una funzione

- Prima di eseguire una chiamata a una funzione, dobbiamo informare il compilatore circa il tipo degli argomenti che essa accetta e il tipo di dato che essa restituisce
- Tali informazioni vengono comunicate al compilatore mediante il **prototipo**
- Esempio:

```
double square(double);
```

- In questo modo il compilatore sa che intendiamo usare una funzione `square` che accetta un argomento di tipo `double` e restituisce un valore di tipo `double`
- È bene inserire il prototipo una sola volta, **prima** della funzione `main`

Funzioni

Definizione e prototipo

- Un listato C nel quale si utilizzano le funzioni contiene di norma sia i prototipi che le definizioni di esse

```
#include <stdio.h>

double square(double);

int main() {
    double z;

    printf("Insert number: ");
    scanf("%lf",&z);
    printf("The square of %lf is %lf\n",z,square(z));
}

double square(double x) {
    return(x*x);
}
```

Funzioni

Passaggio degli argomenti per valore

- Quando passiamo alla funzione `square` la variabile `z`, stiamo passando alla funzione solo il **valore** di `z`
- La funzione **non** modifica il valore di `z`

```
double y, z;  
  
z=5;  
y=square(z);  
printf("%lf", z); /* prints 5 */
```

- Si parla in questo caso di passaggio degli argomenti per valore
- Le cose non cambiano se l'argomento ha lo stesso nome della variabile passata alla funzione

Funzioni

Passaggio degli argomenti per indirizzo

- Alla funzione viene passato un **puntatore** come argomento
- Se il puntatore punta una variabile e la funzione modifica il valore puntato, il valore della variabile risulta modificato al termine della funzione
- Esempio:

```
void swap(int* a, int* b)
{
    int tmp;
    tmp=*a; *a=*b; *b=tmp;
    return;
}
```

- Se la funzione `swap` viene chiamata mediante l'istruzione `swap(&x, &y);` (dove `x` e `y` sono intere), al termine della funzione i valori delle due variabili risultano scambiati

Outline

- 1 Introduzione
- 2 Variabili e costanti
- 3 Input/output di base
- 4 Rappresentazione binaria dei dati
- 5 Operatori
- 6 Conversioni di tipo
- 7 Array e puntatori
- 8 Funzioni
- 9 Controllo del flusso**
- 10 Input/output da file
- 11 Variabili aleatorie in C
- 12 Numeri complessi in C
- 13 Elaborazione in virgola fissa



Flusso di un programma

- Programma C: insieme di istruzioni che, se non specificato altrimenti, vengono eseguite in modo **sequenziale**
- Istruzione: riga di codice terminata dal carattere ;
- Spesso accade di dover modificare la sequenza con cui le istruzioni vengono eseguite
 - ▶ Esempio: **ripetere** un certo blocco di istruzioni finché una condizione rimane vera
 - ▶ Esempio: **selezionare** quale blocco di istruzioni eseguire in relazione al verificarsi di una certa condizione
- In ciascuno degli esempi precedenti occorre specificare la condizione sotto la quale il flusso sequenziale di esecuzione delle istruzioni deve essere modificato
- A tale scopo si possono utilizzare gli **operatori relazionali** e gli **operatori logici**

Operatori relazionali

- Gli operatori relazionali sono: < (minore di), <= (minore di o uguale a), > (maggiore di), >= (maggiore di o uguale a), == (uguale a), != (diverso da)
- Un'espressione relazionale è vera se assume **qualunque** valore diverso da zero, ed è falsa se il suo valore è zero
- L'operatore relazionale == **non va confuso** con l'operatore di assegnamento =

```
int i=4, j=5;
int k;

k=(i==j);    /* k=0 */
k=(i<j);     /* k=1 */
```

Operatori logici

- In C esistono tre operatori logici: `&&` (AND logico), `||` (OR logico), `!` (NOT logico)
 - ▶ Precedenze: `!` $\rightarrow$ `&&` $\rightarrow$ `||` (possibile uso di parentesi)
- Un'espressione logica è vera se assume **qualunque** valore diverso da zero, ed è falsa se il suo valore è zero
- Se un'espressione logica è vera, gli operatori logici restituiscono 1, altrimenti restituiscono 0
- Espressione logica `condizione1 && condizione2`: vera se `condizione1` e `condizione2` sono **entrambe** vere, falsa altrimenti
- Espressione logica `condizione1 || condizione2`: vera se **almeno una** tra `condizione1` e `condizione2` è vera, falsa altrimenti
- Espressione logica `!condizione`: vera se `condizione` è falsa, falsa altrimenti

Operatori logici

Esempi

```
int n; char c=')';  
n = !((!c)&&0x0E);  
printf("%d\n",n); /* prints 1 */
```

```
int x=1, y=1, z=1;  
  
++x || !(--y) && ++z;  
printf("x=%d y=%d z=%d\n",x,y,z);  
printf("%d\n",!y&&z);  
/* prints x=2 y=1 z=1 and then prints 0
```

Due to priority rules, the expression is equivalent to `++x || (!(--y) && ++z)`; the first operand (x) is increased by 1 before being used, it evaluates to 2 and the logical OR evaluates to 1 without processing the second operand;

Before `!y&&z` is evaluated we still have `y=1` and `z=1`; due to priority rules `!y` evaluates to 0 and so does the whole expression `*/`

Decisioni

Espressione condizionale

- Viene scelta una fra due espressioni possibili in base al fatto che una terza espressione sia falsa (valore zero) oppure vera (valore diverso da zero)
- È a tutti gli effetti un'espressione: può stare, ad esempio, a destra di un operatore di **assegnamento**
- Sintassi:

```
espr1 ? espr2 : espr3;
```

- Viene per prima cosa calcolata l'espressione `espr1`
- `espr1` falsa (il risultato è zero): viene calcolata l'espressione `espr3`, il cui valore rappresenta il risultato dell'intera espressione condizionale
- `espr1` vera (il risultato è diverso da zero): viene calcolata l'espressione `espr2`, il cui valore rappresenta il risultato dell'intera espressione condizionale

```
int z = (a > b) ? a : b;      /* z = max{a,b} */  
const int n=(x!=0) ? -1 : 1;
```

Decisioni

Costrutto `if-else`

- Utilizzato per prendere decisioni. Sintassi generale:

```
if (espr1)
    istruzione_1;
else if (espr2)      /* optional */
    istruzione_2;
...
else                /* default case, optional */
    istruzione_n;
```

- Le varie espressioni sono calcolate in ordine
- Quando viene trovata un'espressione vera, viene eseguita la corrispondente istruzione e il processo termina
- Ciascuna istruzione può essere **composta**, cioè un blocco di istruzioni (eseguite in sequenza) fra parentesi graffe { }



Decisioni

Costrutto `switch`

- Alternativo a una catena `if-else` nel caso in cui l'espressione `espr` assume un valore **intero** fra un insieme finito di possibili valori interi costanti
- Sintassi:

```
switch(espr) {  
    case cost_int_1:  
        { istruzione_1; [break;] }  
    ...  
    case cost_int_n:  
        { istruzione_n; [break;] }  
    default:  
        istruzione_def; }
```

- L'espressione intera `espr` viene calcolata; per ogni caso, se il valore di `espr` è uguale alla relative costante, viene eseguita la corrispondente istruzione
- Ciascuna istruzione può essere **composta**
- Possibile uso dell'istruzione `break` **[senza break tutte le istruzioni successive alla prima verificata vengono eseguite]**

Cicli

Costrutto `while`

- Sintassi:

```
while (espr)
    istruzione;
```

- L'espressione `espr` viene calcolata
- Se essa è vera (valore diverso da zero) l'istruzione viene eseguita e l'espressione `espr` viene ri-calcolata
- Il ciclo continua fino a quando il valore di `espr` non diventa zero
- L'istruzione può essere **composta**
- Nel caso in cui l'espressione `espr` non assume mai il valore zero, si rimane "intrappolati" in un loop infinito

```
while(1) {}          /* infinite loop */
```

Cicli

Costrutto `do-while`

- Sintassi:

```
do
    istruzione;
while espr;
```

- Nel caso del costrutto `while` l'espressione `espr` viene calcolata all'inizio del loop
- Se essa è falsa, l'istruzione non viene mai eseguita
- Nel caso del costrutto `do-while` l'istruzione viene sempre eseguita almeno una volta, perché l'espressione `espr` è calcolata alla fine del loop

```
while(0) {n++;}           /* n is never increased */
do {n++;} while(0);       /* n is increased once */
```



Cicli

Costrutto for

- Sintassi:

```
for (espr1 ; espr2 ; espr3)
    istruzione;
```

- Equivalente a:

```
espr1;
while (espr2) {
    istruzione;
    espr3; }
```

- Solitamente `espr1` e `espr3` sono assegnamenti e `espr2` è un'espressione relazionale
- L'istruzione può essere **composta**; il costrutto `for` è utile quando un'istruzione deve essere eseguita un numero **prefissato** di volte

```
int i,n; int s=0;
for (i=0 ; i<5 ; i++) {
    scanf("%d",&n); s+=n; } /* accum. 5 integers */
```

Cicli

Istruzioni `break` e `continue`

- Istruzione `break`: consente di forzare l'**uscita da un loop**
- Utile nel caso si voglia eseguire un'istruzione (eventualmente composta) un numero indefinito di volte, finché non si verifica un condizione di "break out"

```
char k;  
for( ; ; ) {  
    scanf("%c", &k);  
    if(k=='e') break;  
    printf("%c", k); }
```

- Istruzione `continue`: consente di **interrompere l'iterazione corrente** di un loop
- Non forza l'uscita dal loop

```
float f; /* remember to include <math.h> library */  
for( ; ; ) {  
    scanf("%f", &f);  
    if (f<0) continue;  
    else printf("Square root: %f", sqrt(f));  
}
```



Esempio

Fattoriale di un intero positivo

```
#include <stdio.h>
int main() {
    int n, i;
    unsigned long long factorial = 1;

    printf("Enter an integer:");
    scanf("%d", &n);

    if (n<0) {
        printf("Go to study\n");
        return 1;
    }
    else {
        for(i=1; i<=n; i++) factorial *= i;
        printf("Factorial of
               %d=%llu\n", n, factorial);
        return 0;
    }
} /* Includes also the case n=0 */
```



Esempio

Intero positivo pari o dispari

```
#include <stdio.h>
int main() {
    long int n;
    do {
        printf("Enter a positive integer: ");
        scanf("%ld", &n);
    }
    while(n<=0);

    switch(n%2) {
        case 0:
            printf("%ld is even\n", n); break;
        case 1:
            printf("%ld is odd\n", n); break;
    }
    return 0;
}
```

Esempio

Creazione dinamica di una matrice 3×4 di interi

```
#include <stdio.h>
#include <stdlib.h>
#define rowNum 3
#define colNum 4

int main() {
    int i, j, **table;

    table=(int**)malloc(rowNum*sizeof(int*));
    for(i=0; i<rowNum; i++) {
        table[i]=(int*)malloc(colNum*sizeof(int));
    }

    for(i=0; i<rowNum; i++)
        for(j=0; j<colNum; j++) {
            printf("Insert element in position (%d,%d)", i, j);
            scanf("%d", &table[i][j]);
        }

    /* some processing */

    for(i=0; i<rowNum; i++) free(table[i]);
    free(table);
}
```

Outline

- 1 Introduzione
- 2 Variabili e costanti
- 3 Input/output di base
- 4 Rappresentazione binaria dei dati
- 5 Operatori
- 6 Conversioni di tipo
- 7 Array e puntatori
- 8 Funzioni
- 9 Controllo del flusso
- 10 Input/output da file**
- 11 Variabili aleatorie in C
- 12 Numeri complessi in C
- 13 Elaborazione in virgola fissa



Funzione `fopen`

- Spesso i programmi lavorano su grandi quantità di dati; tali dati si trovano su file che devono essere letti e scritti
- Prima di poter lavorare sul file, il programma deve **aprire** il file
- In C, quando un file viene aperto viene creata una struttura dati il cui tipo di dato è `FILE`
- L'apertura del file avviene mediante la funzione **`fopen`**
- Prototipo:

```
FILE *fopen(const char *filename, const char *mode);
```

- `filename`: stringa che contiene il nome del file
- `mode`: stringa che specifica come il file deve essere aperto
 - ▶ “r”: read-only; “w”: write-only; “a”: append
- Restituisce un **puntatore a `FILE`** che deve essere passato come argomento a qualunque funzione lavori sul file
- Restituisce **`NULL`** quando l'apertura del file non è andata a buon fine

Funzione `fclose`

- Quando un file non viene più utilizzato in un programma, esso va chiuso mediante la funzione `fclose`
- Prototipo:

```
int fclose(FILE *);
```

- Esempio di apertura e chiusure di un file `myFile.dat`:

```
FILE *stream;

if ((stream=fopen("myFile.dat", "r"))==NULL) {
    printf("Cannot open file\n");
    exit(EXIT_FAILURE);
}

fclose(stream); /* exit and EXIT_FAILURE defined
                in stdlib.h */
```

Input/output di caratteri

- È possibile leggere un **singolo carattere** da un file aperto in lettura mediante la funzione `fgetc`:

```
int fgetc(FILE *);
```

- Legge il successivo carattere dal file puntato dall'argomento e lo restituisce come intero
- Restituisce il valore speciale **EOF** quando si verifica un errore oppure quando si raggiunge la fine del file
- Analogamente, la funzione `fputc` consente di scrivere un **singolo carattere** sul file puntato dall'argomento
- Restituisce (come intero) il carattere appena scritto

```
int fputc(int, FILE *);
```

Condizione di fine file

Funzione `feof`

- Quando `fgetc` restituisce EOF, non sappiamo se si è verificato un errore oppure se è stata raggiunta la fine del file
- Per risolvere l'ambiguità si può utilizzare la funzione `feof`:

```
int feof(FILE *);
```

- Essa restituisce un intero diverso da zero se è stata raggiunta la fine del file, e zero altrimenti

```
#include <stdio.h>
int main()
{
    FILE *fp=fopen("myFile.txt", "r"); int ch=fgetc(fp);
    while (ch!=EOF) { putchar(ch);
                    ch=fgetc(fp);
                }
    if (feof(fp)) printf("End of file reached.\n");
    else printf("Something wrong.");
    fclose(fp);
    return 0;
}
```



Input/output formattato

Funzioni `fscanf` e `fprintf`

- Per l'input/output formattato da file si utilizzano le funzioni `fscanf` e `fprintf`
- Sono identiche a `scanf` e `printf`, ad eccezione di un argomento aggiuntivo, di tipo puntatore a `FILE`
- Prototipi:

```
int fscanf(FILE *, const char *format, arg_list);  
int fprintf(FILE *, const char *format, arg_list);
```

- `fscanf` restituisce il numero delle variabili a cui un valore è stato assegnato oppure EOF in caso di errore o fine del file
- `fprintf` restituisce il numero di caratteri scritti oppure un valore negativo in caso di errore.

Outline

- 1 Introduzione
- 2 Variabili e costanti
- 3 Input/output di base
- 4 Rappresentazione binaria dei dati
- 5 Operatori
- 6 Conversioni di tipo
- 7 Array e puntatori
- 8 Funzioni
- 9 Controllo del flusso
- 10 Input/output da file
- 11 Variabili aleatorie in C**
- 12 Numeri complessi in C
- 13 Elaborazione in virgola fissa



Funzione `rand`

Valori restituiti

- La libreria standard `stdlib.h` del linguaggio C mette a disposizione la funzione `rand()`, il cui prototipo è

```
int rand(void);
```

- Tale funzione restituisce un numero intero compreso fra 0 e un valore indicato con `RAND_MAX` (0 e `RAND_MAX` inclusi)
- Il valore di `RAND_MAX` dipende dal compilatore; esso è comunque sempre almeno pari a $2^{15}-1=32767$
- Esempio: `RAND_MAX=231-1=2147483647`

Funzione rand

Generazione di interi pseudo-casuali

- L'intero restituito dalla funzione `rand()` è generato in maniera **pseudo-casuale**
- Tale intero è distribuito in modo **uniforme** tra 0 e `RAND_MAX`
- Mediante chiamate successive alla funzione `rand()` è possibile generare una sequenza pseudo-casuale di interi

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int i;
    for(i=0 ; i<5 ; i++) printf("%d ",rand());
    return 0;
}

/* Might print:
16807 282475249 1622650073 984943658 1144108930 */
```




Funzione rand

Modifica del seme

- Se il precedente programma viene mandato in esecuzione più volte, esso genera sempre la **medesima** sequenza di interi pseudo-casuali
- Questo perché l'algoritmo di generazione degli interi pseudo-casuali utilizza un **"seme"** che viene sempre inizializzato allo stesso valore
- È tuttavia possibile modificare il seme utilizzando la funzione **srand**, definita nella libreria `stdlib`:

```
void srand(unsigned int seed);
```

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int i;
    srand(7);
    for(i=0 ; i<5 ; i++) printf("%d ",rand());
    return 0;
}
/* Might print:
117649 1977326743 621132276 452154665 1566311569 */
```



Funzione `time`

- Spesso il seme viene impostato in base all'orologio, mediante la funzione `time`, definita nella libreria `time.h`:

```
time_t time(time_t *timer);
```

- Lo standard C definisce genericamente `time_t` come un tipo di dato aritmetico “adatto” per il tempo di sistema
- Dipende dalla architettura: solitamente un intero (spesso a 64 bit, con o senza segno) ma anche un floating-point
- Tipicamente la funzione `time` restituisce il numero di secondi che sono passati a partire dalla mezzanotte di Greenwich del 01/01/1970
- Se chiamata con un puntatore `timer` a `time_t` diverso da `NULL`, il valore restituito viene memorizzato in `*timer`

Esempio

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {
    printf("Seconds since UNIX epoch:
           %lu\n",time(NULL));
    printf("Minutes: %lu\n",time(NULL)/60);
    printf("Hours: %lu\n",time(NULL)/(60*60));
    printf("Days: %lu\n",time(NULL)/(60*60*24));
    printf("Years: %lu\n",time(NULL)/(60*60*24*365));
    return 0;
}

/* Seconds since UNIX epoch: 1494531875
   Minutes: 24908864
   Hours: 415147
   Days: 17297
   Years: 48 */
```

Esempio

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {
    printf("%d\n", rand());
    srand((unsigned)time(NULL));
    printf("%d\n", rand());
    return 0;
}

/*
Any time the program is run, the first printed value
doesn't change while the second one does
*/
```



Generazione di variabili aleatorie

- A partire dalla funzione `rand()` possiamo generare i campioni di V.A. con distribuzioni qualunque.
- Nel seguito analizzeremo come generare:
 - ▶ V.A. intere uniformi in un intervallo;
 - ▶ V.A. reali uniformi in un intervallo;
 - ▶ V.A. Gaussiane con il metodo di Marsaglia e Bray;

V.A. intere uniformi in un intervallo

`rand() % N` può essere sbagliato

- Obiettivo: generare un intero distribuito uniformemente nell'insieme $\{0, 1, \dots, N - 1\}$
- La tentazione comune è utilizzare `rand() % N`.
- Tuttavia, se $M = \text{RAND_MAX} + 1$ non è multiplo di N :
 - ▶ Alcune classi modulo N compaiono più spesso ("modulo-bias").
 - ▶ Alcuni valori hanno probabilità $\lfloor M/N \rfloor / M$ e altri $\lceil M/N \rceil / M$.
- **Esempio:** se $\text{RAND_MAX} = 10$ e $N = 3$, la mappatura `rand() % 3` corrisponde a

$$\{0, 3, 6, 9\} \mapsto 0 \quad \{1, 4, 7, 10\} \mapsto 1 \quad \{2, 5, 8\} \mapsto 2$$

e quindi

$$\mathbb{P}(0) = \mathbb{P}(1) = 4/11 \quad \mathbb{P}(2) = 3/11$$

- Se RAND_MAX è grande, l'errore può essere piccolo (esempio, dell'ordine di 10^{-9} se $\text{RAND_MAX} = 2^{31} - 1$), tuttavia è sistematico.

Interi uniformi in un intervallo

Rejection sampling (senza modulo-bias)

- Idea: scartare i valori di `rand()` nella “coda” che renderebbe la probabilità non uniforme.

```
#include <stdlib.h>

/* V.A. discreta uniforme in {0,...,n-1} senza
   modulo-bias */

int rand_int(int n)
{
    int r;
    int limit = (RAND_MAX / n) * n; /* massimo
                                     multiplo di n <= RAND_MAX */
    do {
        r = rand();
    } while (r >= limit);
    return r % n;
}
```

Generazione di V.A. uniformi reali

Intervalli continui

- Spesso è necessario generare **V.A. uniformi a valori reali**.
- A partire da `rand()`, che genera interi uniformi in

$$\{0, 1, \dots, \text{RAND_MAX}\},$$

possiamo costruire una V.A. uniforme in un intervallo reale mediante una trasformazione lineare.

- Caso base: V.A. uniforme in $(0, 1)$.
- La prossima slide introduce la funzione `randu`, che genera una V.A. uniforme in $(0, 1)$ e la funzione `rand_uniform`, che genera una V.A. uniforme in un intervallo (a, b) generico.

Generazione di V.A. uniformi reali

Esempi in C

```
#include <stdlib.h>

/* Uniform random variable in (0,1) */
double randu(void)
{
    return ((double)rand() + 1.0)
           / ((double)RAND_MAX + 2.0);
}

/* Uniform random variable in (a,b) */
double rand_uniform(double a, double b)
{
    return a + (b - a) * randu();
}
```



Generazione di V.A. Gaussiane

Metodo di Marsaglia e Bray

- Siano $U_1 \sim \mathcal{U}(-1, 1)$ e $U_2 \sim \mathcal{U}(-1, 1)$ due V.A. uniformi nell'intervallo $[-1, 1]$
- Siano u_1 e u_2 realizzazioni di U_1 e U_2
- Il seguente algoritmo genera realizzazioni di due V.A. Gaussiane indipendenti $X_1 \sim \mathcal{N}(0, 1)$ e $X_2 \sim \mathcal{N}(0, 1)$

1: **repeat**

2: generate u_1

3: generate u_2

4: $d \leftarrow u_1^2 + u_2^2$

5: **until** $0 < d < 1$

6: $f = \sqrt{-2 \ln(d)/d}$

7: $x_1 \leftarrow (f \times u_1)$

8: $x_2 \leftarrow (f \times u_2)$

Generazione di V.A. Gaussiane

Esempio

```
#include <math.h>
#include <stdlib.h>
double randn(double mu, double sigma)
{
    double u1,u2,d,f;
    static double x1,x2;
    static int flag=0;
    if(flag==1)
    {
        flag = !flag;
        return (mu+sigma*(double)x2); }
    do
    {
        u1 = -1 + ((double) rand() /RAND_MAX) *2;
        u2 = -1 + ((double) rand() /RAND_MAX) *2;
        d = u1*u1+u2*u2;}
    while (d>= 1 || d==0);
    f = sqrt ((-2*log(d))/d);
    x1 = u1*f;
    x2 = u2*f;
    flag=!flag;
    return (mu+sigma*(double)x1);
}
```



V.A. derivate da Gaussiane

Confronto concettuale

- Siano $X_i \sim \mathcal{N}(0, \sigma^2)$ V.A. Gaussiane indipendenti.
- Diverse trasformazioni delle V.A. X_i permettono di ottenere distribuzioni di uso molto comune:
 - ▶ Chi-quadro
 - ▶ Rayleigh / Rice
 - ▶ Maxwell
 - ▶ Lognormale
 - ▶ ...
- Una singola funzione che genera V.A. Gaussiane (`randn`) permette quindi di costruire molte **altre distribuzioni**.

V.A. con Distribuzione di Rayleigh

Definizione

- Siano $X_1 \sim \mathcal{N}(0, \sigma^2)$ e $X_2 \sim \mathcal{N}(0, \sigma^2)$ due V.A. Gaussianhe, a valor medio nullo, varianza σ^2 , indipendenti.
- La V.A. $R = \sqrt{X_1^2 + X_2^2}$ è distribuita alla Rayleigh:

$$f_R(r) = \begin{cases} \frac{r}{\sigma^2} e^{-r^2/(2\sigma^2)} & r \geq 0 \\ 0 & r < 0 \end{cases}$$

- Risulta

$$\mathbb{E}[R] = \sigma \sqrt{\frac{\pi}{2}} \quad \text{e} \quad \mathbb{V}[R] = \frac{4 - \pi}{2} \sigma^2$$

V.A. con Distribuzione di Rice

Definizione

- Siano $X_1 \sim \mathcal{N}(\nu, \sigma^2)$ e $X_2 \sim \mathcal{N}(0, \sigma^2)$ due V.A. Gaussiane indipendenti.
- La V.A.

$$R = \sqrt{X_1^2 + X_2^2}$$

è distribuita alla Rice:

$$f_R(r) = \begin{cases} \frac{r}{\sigma^2} \exp\left(-\frac{r^2 + \nu^2}{2\sigma^2}\right) I_0\left(\frac{r\nu}{\sigma^2}\right) & r \geq 0 \\ 0 & r < 0 \end{cases}$$

dove $I_0(x)$ si chiama funzione di Bessel modificata di prima specie e ordine zero.

- Caso particolare $\nu = 0$: distribuzione di Rayleigh.

V.A di Rayleigh e Rice

Esempio di generazione

- Le due distribuzioni differiscono solo per il valore medio di una Gaussiana

```
#include <math.h>
#include <stdlib.h>

double rand_rayleigh(double sigma)
{
    double x = randn(0.0, sigma);
    double y = randn(0.0, sigma);
    return sqrt(x*x + y*y);
}

double rand_rice(double nu, double sigma)
{
    double x = randn(nu, sigma);
    double y = randn(0.0, sigma);
    return sqrt(x*x + y*y);
}
```

Altre V.A. derivate dalla Gaussiana

Chi-quadro, Maxwell e Lognormale

- Se $X_i \sim \mathcal{N}(0, 1)$ sono V.A. Gaussiane indipendenti, allora

$$Y = \sum_{i=1}^k X_i^2$$

è una V.A. con distribuzione **chi-quadro** con k gradi di libertà.

- Se $X_i \sim \mathcal{N}(0, \sigma^2)$, $i = 1, 2, 3$, sono V.A. Gaussiane indipendenti allora

$$Y = \sqrt{X_1^2 + X_2^2 + X_3^2}$$

è una V.A. con distribuzione di **Maxwell**.

- Se $X \sim \mathcal{N}(\mu, \sigma^2)$ è una V.A. Gaussiana, allora

$$Y = e^X$$

è una V.A. con distribuzione **lognormale**.

Generazione in C

Chi-quadrato, Maxwell e Lognormale

```
#include <math.h>
#include <stdlib.h>

double rand_chi2(int k)
{
    int i; double s = 0.0, x;
    for(i=0; i<k; i++){
        x = randn(0.0, 1.0);
        s += x*x; }
    return s;
}

double rand_maxwell(double sigma)
{
    double x = randn(0.0, sigma), y = randn(0.0, sigma), z =
        randn(0.0, sigma);
    return sqrt(x*x + y*y + z*z);
}

double rand_lognorm(double mu, double sigma)
{
    return exp(randn(mu, sigma));
}
```



Generazione di V.A. via inversione della CDF

Principio generale

- Sia $U \sim \mathcal{U}(0, 1)$ una variabile aleatoria uniforme
- Sia F_X la funzione di distribuzione cumulativa (CDF) di una V.A. continua X
- Si assuma che F_X sia strettamente crescente e invertibile
- Definendo

$$X = F_X^{-1}(U)$$

si ottiene una variabile aleatoria X con funzione di distribuzione F_X

- In conclusione, per **generare una V.A. di cui si conosce l'inversa della CDF**, F_X^{-1} , si può applicare la funzione F_X^{-1} ad una V.A. uniforme in $(0, 1)$

Generazione di V.A. via inversione della CDF

Dimostrazione

- Vogliamo mostrare che la V.A. $X = F_X^{-1}(U)$ ha CDF F_X
- Per ogni x nel dominio di X :

$$\mathbb{P}(X \leq x) = \mathbb{P}(F_X^{-1}(U) \leq x)$$

- Poiché F_X è crescente, possiamo applicare F_X a entrambi i termini della disequazione al secondo membro:

$$\mathbb{P}(X \leq x) = \mathbb{P}(U \leq F_X(x))$$

- Essendo $U \sim \mathcal{U}(0, 1)$:

$$\mathbb{P}(U \leq F_X(x)) = F_X(x)$$

- Quindi X ha funzione di distribuzione cumulativa F_X

□

Generazione via inversione della CDF

Esempio: V.A. esponenziale in C

- Utilizziamo la funzione `randu` precedentemente definita (esclude lo 0 per evitare $\log(0)$)

```
#include <stdlib.h>
#include <math.h>

/* U: Uniform random variable in (0,1) */
double randu(void)
{
    return ((double)rand() + 1.0) / ((double)RAND_MAX
    + 2.0);
}

/* X ~ Exp(lambda), lambda > 0 */
double rande(double lambda)
{
    double u = randu();
    return -log(u) / lambda;
}
```



Distribuzione esponenziale

Verifica empirica (media e varianza)

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <time.h>

int main(void)
{
    const int N = 1000000;
    const double lambda = 2.0;
    double x, m=0.0, s2=0.0;
    srand((unsigned)time(NULL));

    for(int i=0; i<N; i++){
        x = rande(lambda);
        m += x; s2 += x*x;
    }
    m /= N; s2 = (s2 / N) - m*m;

    printf("mean ~ %.6f (theory %.6f)\n", m, 1.0/lambda);
    printf("var ~ %.6f (theory %.6f)\n", s2,
        1.0/(lambda*lambda));
    return 0;
}
```

Outline

- 1 Introduzione
- 2 Variabili e costanti
- 3 Input/output di base
- 4 Rappresentazione binaria dei dati
- 5 Operatori
- 6 Conversioni di tipo
- 7 Array e puntatori
- 8 Funzioni
- 9 Controllo del flusso
- 10 Input/output da file
- 11 Variabili aleatorie in C
- 12 Numeri complessi in C**
- 13 Elaborazione in virgola fissa

Tipi di dato complessi in C

Header, tipi e unità immaginaria

- Header file: `#include <complex.h>`
- Tipi complessi disponibili:
 - ▶ `float complex` (precisione decimale 6–9)
 - ▶ `double complex` (precisione decimale 13–15)
 - ▶ `long double complex` (precisione estesa, dipendente dalla piattaforma)
- Nel seguito useremo principalmente `double complex`
- Unità immaginaria: la macro `I`

```
#include <complex.h>

double complex z = 1.0 + 2.0*I;
```

Nota: `I` è una macro definita in `<complex.h>`.

Rappresentazione cartesiana

Forma $z = x + jy$

- Un numero complesso può essere scritto nella sua forma cartesiana

$$z = x + jy,$$

dove j è l'unità immaginaria, x è la parte reale e y è la parte immaginaria.

- In C possiamo definire un numero complesso in forma cartesiana.

```
#include <complex.h>

double complex z = 3.0 - 4.0*I;
```

- In C un numero complesso è sempre memorizzato tramite le sue **componenti reale e immaginaria**.

Operatori aritmetici

Operazioni tra numeri complessi

- Sono supportati i normali operatori aritmetici.
- Le operazioni seguono esattamente le regole matematiche.

```
double complex z1 = 1.0 + 2.0*I;  
double complex z2 = 3.0 - 1.0*I;  
  
double complex s = z1 + z2;    // s = 4.0 + 1.0*I  
double complex d = z1 - z2;    // d = -2.0 + 3.0*I  
double complex p = z1 * z2;    // p = 5.0 + 5.0*I  
double complex q = z1 / z2;    // q = 0.1 + 0.7*I (approx)
```

- Le usuali operazioni tra numeri reali e complessi sono anch'esse supportate.

```
double complex z = 1.0 + 2.0*I;  
  
double complex a = 2.0 * z;    // a = 2.0 + 4.0*I  
double complex b = z + 1.0;    // b = 2.0 + 2.0*I  
double complex c = z - 1.0;    // c = 0.0 + 2.0*I  
double complex d = z / 2.0;    // d = 0.5 + 1.0*I
```



Rappresentazione polare

Funzione `cexp`

- Un numero complesso può essere anche scritto nella sua forma polare (r modulo e θ argomento):

$$z = r e^{j\theta}$$

- In C possiamo esprimere un numero complesso in forma polare mediante la funzione `cexp`.
- Se `r` è il modulo e `theta` è l'argomento, possiamo esprimere il numero complesso come `r*cexp(I*theta)`

```
#include <complex.h>
#include <math.h>

double r = 2.0;
double theta = M_PI/4;
double complex z = r * cexp(I * theta);
```

Parti reale e immaginaria, coniugato

Funzioni `creal`, `cimag`, `conj`

- Accesso alle componenti: `creal(z)` e `cimag(z)`
- Coniugato: `conj(z)`

```
#include <stdio.h>
#include <complex.h>

double complex z = 3.0 - 4.0*I;

printf("Re = %.3f, Im = %.3f\n", creal(z), cimag(z));
/* prints: Re = 3.000, Im = -4.000 */

/* Conjugate */
double complex zc = conj(z);
printf("Re = %.3f, Im = %.3f\n", creal(zc),
      cimag(zc));
/* prints: Re = 3.000, Im = 4.000 */
```



Modulo e argomento

Funzioni `cabs` e `carg`

- Modulo e argomento di un numero complesso z :

$$|z| = \sqrt{\Re(z)^2 + \Im(z)^2}$$

$$\arg(z) = \text{atan2}(\Im(z), \Re(z))$$

- In C, il modulo e l'argomento di un numero complesso possono essere estratti con le funzioni `cabs` e `carg`.

```
double complex z = 3.0 - 4.0*I;  
double r = cabs(z);    // r = 5.0  
double phi = carg(z);  // phase = -0.927 (rad)
```



Forma cartesiana e polare

Esempio completo

```
#include <stdio.h>
#include <complex.h>
#include <math.h>

int main(void) {
    double complex z = 3.0 - 4.0*I;
    printf("z = %.3f + %.3fi\n", creal(z), cimag(z));
    printf("|z| = %.3f, arg(z) = %.3f\n", cabs(z), carg(z));
    // prints: z = 3.000 + -4.000i
    //          |z| = 5.000, arg(z) = -0.927

    double r = 2.0;
    double theta = M_PI/6;
    double complex w = r * cexp(I * theta);
    printf("w = %.3f + %.3fi\n", creal(w), cimag(w));
    // prints: w = 1.732 + 1.000i

    return 0;
}
```

Esempio: Campionamento di un fasore

Dal modello continuo all'implementazione in C

- Fasore di ampiezza A , frequenza f_0 e fase iniziale ϕ :

$$x(t) = A e^{j(2\pi f_0 t + \phi)}$$

- Esempio di estrazione in C del campione complesso di un fasore all'istante nT_s ($T_s = 1/f_s$ periodo di campionamento):

```
double A = 1.0;           // amplitude
double f0 = 1000.0;        // frequency (Hz)
double phi = 0.0;          // initial phase
double Fs = 3000.0;        // sampling frequency (Hz)

double complex x =
    A * cexp(I * (2.0*M_PI*f0*n/Fs + phi));
```

Stampa di numeri complessi

`printf` non stampa direttamente i complessi

- Le funzioni `printf` e `fprintf` non hanno uno specificatore per `double complex`
- Occorre stampare parte reale e parte immaginaria utilizzando `creal` e `cimag`

```
double complex z = 1.234 + 5.678*I;

printf("z = %.4f + %.4fi\n", creal(z), cimag(z));
// prints 1.2340 + 5.6780i

printf("mod = %.6f, arg = %.6f\n",
       cabs(z), carg(z));
// prints mod = 5.810546, arg = 1.356794
```

Salvataggio di vettori complessi

Esportazione in formato CSV

- Per analizzare molti campioni complessi è utile **salvarli su file**.
- Scrivere due colonne: parte reale e parte immaginaria.
- Il formato CSV è facilmente importabile in Python e Matlab.

```
FILE *fp;
if ((fp=fopen("data.csv", "w"))==NULL) {
    printf("Cannot open file\n");
    exit(EXIT_FAILURE);
}
// x: vector of double complex of length N
for (int n = 0; n < N; n++) {
    fprintf(fp, "%.10f,%.10f\n", creal(x[n]), cimag(x[n]));
}

fclose(fp);
```

Esempio di lettura da Python:

```
import numpy as np
data = np.loadtxt("data.csv", delimiter=",")
z = data[:,0] + 1j*data[:,1]
```


Applicazione: DFT in C

Definizione e complessità

- Dato un vettore di numeri complessi $\mathbf{x}$ con elementi $x[n]$, $n = 0, \dots, N - 1$, la DFT di $\mathbf{x}$ è il vettore $\mathbf{X}$ i cui elementi $X[k]$, $k = 0, \dots, N - 1$, sono definiti come:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}, \quad k = 0, \dots, N - 1.$$

- Definendo la costante di rotazione (“twiddle factor”) $W_N = e^{-j2\pi/N}$, si ottiene forma compatta

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn}, \quad k = 0, \dots, N - 1.$$

- **Costo computazionale:** N coefficienti, ciascuno con N moltiplicazioni $\Rightarrow \mathcal{O}(N^2)$.

Applicazione: DFT in C

Implementazione con `double complex`

```
#include <complex.h>
#include <math.h>

void dft(const double complex *x, double complex *X,
        int N)
{
    for (int k = 0; k < N; k++) {
        double complex s = 0.0 + 0.0*I;
        for (int n = 0; n < N; n++) {
            double theta = -2.0*M_PI*k*n / (double)N;
            s += x[n] * cexp(I * theta);
        }
        X[k] = s;
    }
}
```

- `x`: ingresso (vettore di N campioni complessi), `X`: uscita.
- Per segnali reali dovremo porre `cimag(x[n])` sarà nullo.



Interpretazione della DFT

Valutazione numerica dello spettro

- Consideriamo un segnale $x(t)$ identicamente nullo per $t \notin [0, T]$.
- $x(t)$ potrebbe essere ottenuto, ad esempio, finestrando in $[0, T]$ un altro segnale.
- $x(t)$ è trasformabile, con trasformata $X(f)$.
- Campioniamo $x(t)$ in $[0, T]$ con passo di campionamento $\Delta t = T/N$ ($F_s = N/T$), ottenendo il vettore $\mathbf{x}$.
- In frequenza, il campionamento determina la ripetizione periodica di $X(f)$, con periodo F_s : $\tilde{X}(f) = (1/\Delta t) \sum_{k=-\infty}^{\infty} X(f - kF_s)$.
- Il vettore $\mathbf{X} = \text{DFT}(\mathbf{x})$ contiene i campioni di $\tilde{X}(f)$ alle frequenze $k\Delta f$, $k = 0, \dots, N-1$.
- Se F_s è tale da eliminare effetti di aliasing ($F_s \geq 2B$, dove B è la banda di $x(t)$) allora $\Delta t \cdot \mathbf{X}$ fornisce i campioni dello spettro di $x(t)$ campionato con passo $\Delta f = 1/T$ sull'intervallo di frequenze $[0, F_s]$.



Interpretazione della DFT

Osservazioni

- L'intervallo T sul quale viene osservato e campionato il segnale nel dominio del tempo determina la risoluzione nel dominio della frequenza:

$$\Delta f = \frac{1}{T} .$$

- Il passo di campionamento nel tempo, $\Delta t = 1/F_s$ determina l'ampiezza dell'intervallo di frequenze osservabile dalla DFT:

$$[0, F_s] = [0, 1/\Delta t] .$$

Outline

- 1 Introduzione
- 2 Variabili e costanti
- 3 Input/output di base
- 4 Rappresentazione binaria dei dati
- 5 Operatori
- 6 Conversioni di tipo
- 7 Array e puntatori
- 8 Funzioni
- 9 Controllo del flusso
- 10 Input/output da file
- 11 Variabili aleatorie in C
- 12 Numeri complessi in C
- 13 Elaborazione in virgola fissa**

Elaborazione numerica di un segnale

Floating point vs fixed point

- Consideriamo la catena di elaborazione in digitale di un segnale $x(t)$:

$$x(t) \rightarrow \text{ADC} \rightarrow x[n] \rightarrow \text{Processing} \rightarrow y[n] \rightarrow \text{DAC} \rightarrow y(t)$$

- I campioni $x[n]$ sono rappresentati su b bit.
- L'elaborazione digitale può essere effettuata:
 - ▶ in **hardware** (FPGA, ASIC);
 - ▶ in **software** (CPU), tipicamente in linguaggio C.
- Nel caso di elaborazione software in C (sistemi embedded), i campioni possono essere rappresentati:
 - ▶ in **virgola mobile** (`float`, `double`);
 - ▶ in **virgola fissa** (come vedremo, tipi di dato interi).



Rappresentazione in virgola mobile

Rappresentazione in C

- La rappresentazione binaria in virgola mobile di un numero decimale è quella mediante **segno**, **mantissa** ed **esponente**.
- La virgola viene sempre posta dopo la prima cifra significativa (che nel caso binario è sempre un bit **1**):

$$f = (-1)^{b_{31}} \times (1.b_{22} \cdots b_0)_2 \times 2^{(b_{30} \cdots b_{23})_2 - 127}$$

- A meno dell'offset sull'esponente, essa equivale alla **notazione scientifica**.
- Tipi di dato floating point in C (vedere slide 23–28):
 - ▶ `float` (formato “binary 32”),
 - ▶ `double` (formato “binary 64”).
 - ▶ `long double` (formato “binary 128”).

Rappresentazione in virgola fissa

Idea di base

- Nella rappresentazione in virgola fissa, si decide a priori:
 - ▶ il numero di bit da dedicare alla **parte intera**, m ;
 - ▶ il numero di bit da dedicare alla **parte frazionaria**, n .
- **Svantaggi:**
 - ▶ range rappresentabile ridotto;
 - ▶ minore risoluzione.
- **Vantaggi:**
 - ▶ minore costo computazionale e consumo energetico;
 - ▶ unità floating point non sempre presente nei sistemi embedded.



Rappresentazione in virgola fissa

Esempio numerico

- Assumiamo una rappresentazione a 16 bit con $m = 4$ bit per la parte intera e $n = 12$ bit per la parte decimale.
- Consideriamo, ad esempio, il numero decimale

$$x = 3.157$$

- Per ottenere la sua rappresentazione in virgola fissa, convertiamo separatamente la parte intera (3) e la parte decimale (0.157) con i numeri di bit specificati:
- **Parte intera** ($m = 4$ bit): 3 $\rightarrow$ 0011
- Parte decimale ($n = 12$): vedere slide successiva



Rappresentazione in virgola fissa

Esempio numerico

- **Parte decimale** ($n = 12$ bit): **0.157**

$$0.157 \times 2 = 0.314 \rightarrow 0$$

$$0.314 \times 2 = 0.628 \rightarrow 0$$

$$0.628 \times 2 = 1.256 \rightarrow 1$$

$$0.256 \times 2 = 0.512 \rightarrow 0$$

$$0.512 \times 2 = 1.024 \rightarrow 1$$

$$0.024 \times 2 = 0.048 \rightarrow 0$$

$$0.048 \times 2 = 0.096 \rightarrow 0$$

$$0.096 \times 2 = 0.192 \rightarrow 0$$

$$0.192 \times 2 = 0.384 \rightarrow 0$$

$$0.384 \times 2 = 0.768 \rightarrow 0$$

$$0.768 \times 2 = 1.536 \rightarrow 1$$

$$0.536 \times 2 = 1.072 \rightarrow 1$$

$$x \rightarrow 0011001010000011$$

- Rappresentazione di x **approssimata**: la parte decimale è stata troncata a $n = 12$ bit.
- La posizione della virgola è **implicita**.
- Se x è negativo, si converte $|x|$ e si applica il complemento a 2 all'intera parola di $m + n$ bit.



Notazione Q

Formalizzazione della virgola fissa

- Il formato utilizzato nel precedente esempio è detto Q4.12.
- In generale, $Qm.n$ indica un formato a $m + n$ bit con
 - ▶ m bit per la parte intera;
 - ▶ n bit per la parte frazionaria.
- Il valore reale rappresentato è:

$$\hat{x} = \frac{X_{\text{int}}}{2^n}$$

dove X_{int} è l'intero in complemento a 2 corrispondente alla parola di $m + n$ bit.

- Esempio (Q4.12):

$$x = 3.157 \quad x \rightarrow \text{0011001010000011}$$

$$X_{\text{int}} = 12931 \quad \hat{x} = 3.156982$$



Notazione Q

Range rappresentabile e risoluzione

- Nel formato $Qm.n$ (signed), il range di valori rappresentabili è

$$x_{\min} = -2^{m-1} \quad x_{\max} = 2^{m-1} - 2^{-n}$$

- Risoluzione (passo minimo rappresentabile):

$$\Delta = 2^{-n}$$

- Ad esempio, nel formato $Q4.12$ abbiamo

$$x_{\min} = -8 \quad x_{\max} = 8 - 2^{-12} \approx 7.9998$$

$$\Delta = 2^{-12} \approx 2.44 \times 10^{-4}$$

Operazioni in formato Q

Somma e sottrazione

- Siano $x, y \in Qm.n$. Allora:

$$x \pm y \in Qm.n$$

- L'operazione si effettua **come una normale somma**, rappresentando il risultato su $m + n$ bit, con la posizione della virgola che resta invariata.
- **Esempio** (Q4.4):

$$x = 0011.0100 \quad (3.25_{10}) \qquad y = 0001.1100 \quad (1.75_{10})$$

$$\begin{array}{r} 0011.0100 \\ + 0001.1100 \\ \hline 0101.0000 \end{array}$$

$$\longrightarrow x + y = 0101.0000 \quad (5.0_{10})$$

- Possibile overflow se il risultato eccede il range rappresentabile.

Operazioni in formato Q

Moltiplicazione

- Se $x, y \in Qm.n$, allora:

$$x \cdot y \in Q(2m).(2n).$$

- L'operazione si effettua **come una normale moltiplicazione**, ma il numero di bit della parte intera e frazionaria si somma.
- Esempio** (Q4.4):

$$x = 0011.0100 \quad (3.25_{10}) \quad y = 0001.1000 \quad (1.5_{10})$$

$$\begin{array}{r} 0011.0100 \\ \times 0001.1000 \\ \hline 00000100.11100000 \end{array}$$

$$\longrightarrow x \cdot y = 00000100.11100000 \quad (4.875_{10})$$

- Nota:** Rispetto all'addizione, il formato Q di operandi, Q4.4, e risultato, $Q(4 + 4).(4 + 4) = Q(8.8)$, è diverso.



Moltiplicazione in formato Q

Riposizionamento della virgola

- Sappiamo che:

$$x, y \in Qm.n \quad \Rightarrow \quad x \cdot y \in Q(2m).(2n)$$

- Tuttavia, se il risultato deve essere rappresentato nel formato $Qm.n$, occorre definire una funzione che mappi $Q(2m).(2n)$ su $Qm.n$.
- Tale mappatura **non è univoca** e comporta scelte progettuali.
- Nel seguito approfondiamo:
 - ▶ Riposizionamento coerente (fixed-point rescaling);
 - ▶ Scalatura preventiva (headroom);
 - ▶ Scalatura del risultato (extra right-shift).

Moltiplicazione in formato Q

Riposizionamento coerente (fixed-point rescaling)

- **Riposizionamento coerente:** si trasla la rappresentazione di n bit verso destra (compresa la virgola) e si conservano solo gli $m + n$ bit meno significativi.
- Esempio in Q3.5:

$$x = 010.11011 \ (2.84375_{10}) \quad y = 001.00110 \ (1.1875_{10})$$

$$000011.0110000010 \quad Q(6.10)$$

shift di 5 ↓↓

$$0000000011.01100$$

mantengo 8 bit ↓↓

$$011.01100 \quad Q(3.5)$$

- Il risultato, 3.375_{10} contro 3.376953125_{10} , soffre di un errore numerico, dovuto allo shift, sui decimali meno significativi.

Moltiplicazione in formato Q

Effetto di wrap-around

- **Problema:** se il prodotto non è rappresentabile in $Qm.n$, si verifica un errore molto significativo.
- Si tratta di un cosiddetto effetto modulo, o di “wrap-around”.
- Esempio in Q3.5:

$$x = 010.11011 \text{ (} 2.84375_{10} \text{)} \quad y = 011.00110 \text{ (} 3.1875_{10} \text{)}$$

$$00\textcolor{red}{1}001.\textcolor{blue}{000}1000010 \quad Q(6.10)$$

shift di 5 ↓

$$0000000\textcolor{red}{1}001.\textcolor{blue}{000}10$$

mantengo 8 bit ↓

$$\textcolor{blue}{001}.\textcolor{blue}{000}10 \quad Q(3.5)$$

- Nel caso specifico, si ottiene 1.0625_{10} contro 9.064453125_{10} .

Rappresentazione di fixed-point in C

Rappresentazione con tipi interi

- In C non esiste il tipo di dato fixed-point.
- Però la posizione della virgola è **implicita**: possiamo allora ignorare la virgola e memorizzare un numero in formato $Qm.n$ su un **intero** a $m + n$ bit.

- Indicando l'intero con X_{int} , il valore rappresentato è

$$x = \frac{X_{\text{int}}}{2^n}.$$

- Si usano tipi interi a bit garantiti (libreria `stdint.h`), come:
 - ▶ `int16_t` (16 bit), `int32_t` (32 bit), `int64_t` (64 bit).
- **Esempio**: il valore $1.1875_{10} = 001.00110$ (Q3.5) può essere memorizzato come $00100110 = 38$ in una variabile `int8_t`.

Moltiplicazione e riposizionamento coerente

Implementazione C (`int8_t`)

- Funzione `qmul8`: moltiplicazione fixed-point tra fattori `int8_t` e risultato `int8_t`, con n bit per la parte frazionaria.

```
#include <stdint.h>

int8_t qmul8(int8_t X_int, int8_t Y_int, int n)
{
    int16_t prod = (int16_t)X_int * (int16_t)Y_int;

    int8_t Z_int = (int8_t)(prod >> n);
    /* fixed-point rescaling */

    return Z_int;
}
```

Moltiplicazione e riposizionamento coerente

Implementazione C (`int16_t`)

- Funzione `qmul16`: moltiplicazione fixed-point tra fattori `int16_t` e risultato `int16_t`, con n bit per la parte frazionaria.

```
#include <stdint.h>

int16_t qmul16(int16_t X_int, int16_t Y_int, int n)
{
    int32_t prod = (int32_t)X_int * (int32_t)Y_int;

    int16_t Z_int = (int16_t)(prod >> n);
    /* fixed-point rescaling */

    return Z_int;
}
```

Esempio

Esempio coerente con Q3.5 (senza wrap-around)

- Nel nostro esempio Q3.5:

$$x = 010.11011 \text{ (} 2.84375_{10} \text{)} \quad y = 001.00110 \text{ (} 1.1875_{10} \text{)}$$

$$\Rightarrow X_{\text{int}} = 91, \quad Y_{\text{int}} = 38, \quad n = 5.$$

```
#include <stdint.h>

int8_t  X = 91;           /* x=2.84375 in Q3.5 */
int8_t  Y = 38;           /* y=1.1875  in Q3.5 */

int16_t Z = qmul8(X,Y,5); /* Z=108 */
```

- Interpretazione: il valore rappresentato è

$$z = \frac{108}{2^5} = 3.375.$$

Contenimento dell'effetto modulo

Scaling preventivo (headroom) sugli operandi

- Un approccio per ridurre la probabilità di wrap-around consiste nell'introdurre un **fattore di scala preventivo** sugli operandi (headroom):

$$x' = \frac{x}{2^h}, \quad y' = \frac{y}{2^h} \quad \longrightarrow \quad z = \left(\frac{x}{2^h}\right) \cdot \left(\frac{y}{2^h}\right).$$

- **Idea:** ridurre preventivamente la dinamica degli operandi (il ché può comportare perdita di precisione) per aumentare la probabilità che il risultato sia rappresentabile.
- A parte la possibile perdita di precisione, il risultato è affetto da un fattore di scala 2^{-2h} , che però è **costante** (no distorsione) e noto a priori.

Scaling preventivo

Esempio

- Esempio in Q3.5 con $h = 1$:

$$x = 010.11011 \text{ (} 2.84375_{10} \text{)}$$

$$y = 011.00110 \text{ (} 3.1875_{10} \text{)}$$

$$x' = 001.01101 \text{ (} 1.40625_{10} \text{)}$$

$$y' = 001.10011 \text{ (} 1.59375_{10} \text{)}$$

$$000010.0011110111 \text{ Q(6.10)}$$

shift di 5 ↓↓

$$0000000010.00111$$

mantengo 8 bit ↓↓

$$010.00111 \text{ Q(3.5)}$$

- Il risultato è $010.00111_2 = 2.21875_{10}$. Ricordando il fattore di scala, esso corrisponde a $2.21875 \cdot 4 = 8.875$.
- Il risultato corretto (ma non rappresentabile in Q3.5) sarebbe 9.064453125_{10} : c'è stata una perdita di precisione, ma **abbiamo evitato il wrap-around**.

Scaling preventivo

Implementazione C (int16_t)

- Funzione `qmul16_pre`: moltiplicazione fixed-point tra fattori `int16_t` e risultato `int16_t`, con n bit per la parte frazionaria e scaling preventivo.

```
#include <stdint.h>

int16_t qmul16_pre(int16_t X, int16_t Y, int n, int h)
{
    X >>= h; /* headroom */
    Y >>= h; /* headroom */

    int32_t prod = (int32_t)X * (int32_t)Y;

    int32_t z     = prod >> n;
    /* fixed-point rescaling */

    return (int16_t)z;
}
```


Contenimento dell'effetto modulo

Scaling sul risultato (extra right-shift)

- Anziché scalare preventivamente gli operandi, scaliamo il risultato dopo la moltiplicazione:

$$z = (x \cdot y) \cdot \frac{1}{2^p}$$

- Il risultato è affetto da un fattore di scala 2^{-p} , che di nuovo è **costante** (no distorsione) e noto a priori.

Scaling sul risultato

Esempio

- Esempio in Q3.5 con $p = 1$:

$$x = 010.11011 \text{ (} 2.84375_{10} \text{)} \quad y = 011.00110 \text{ (} 3.1875_{10} \text{)}$$

$$001001.0001000010 \quad \text{Q(6.10)}$$

shift di 5 $\Downarrow$

$$00000001001.00010$$

moltiplico per $2^{-p} = 2^{-1} \Downarrow$

$$00000000100.10001$$

mantengo 8 bit $\Downarrow$

$$100.10001 \quad \text{Q(3.5)}$$

- Il risultato è $100.10001_2 = 4.53125_{10}$. Ricordando il fattore di scala, esso corrisponde a $4.53125 \cdot 2 = 9.0625$.
- In questo esempio, **evitiamo il wrap-around** e **aumentiamo la precisione** del risultato rispetto allo scaling preventivo.

Scaling sul risultato

Implementazione C (int16_t)

- Funzione `qmul16_post`: moltiplicazione fixed-point tra fattori `int16_t` e risultato `int16_t`, con n bit per la parte frazionaria e scaling sul risultato.

```
#include <stdint.h>

int16_t qmul16_post(
    int16_t X, int16_t Y, int n, int p)
{
    int32_t prod = (int32_t)X * (int32_t)Y;

    int32_t z     = prod >> (n+p);
    /* fixed-point rescaling + extra right-shift */

    return (int16_t)z;
}
```



Scaling preventivo vs sul risultato

Confronto

- **Scaling preventivo (headroom):**
 - ▶ riduce la dinamica prima della moltiplicazione;
 - ▶ maggiore controllo sul range;
 - ▶ possibile perdita di precisione sugli operandi.
- **Scaling sul risultato (extra right-shift):**
 - ▶ implementazione più semplice;
 - ▶ introduce un fattore di scala costante 2^{-p} ;
 - ▶ maggiore precisione rispetto allo scaling preventivo.
- In entrambi i casi:
 - ▶ si riduce la probabilità di wrap-around;
 - ▶ il segnale viene scalato ma non distorto;
 - ▶ il guadagno complessivo è noto e controllabile.